

深入剖析 Claude Code

1,900 个源文件 · 512,000+ 行 TypeScript

从启动流程到多智能体协调，全面拆解现代 AI 编程助手

SZY

2026 年 6 月 5 日

目录

I	基础架构	3
1	概述与泄露始末	4
1.1	Claude Code 是什么	4
1.2	泄露事件	4
1.2.1	时间线	4
1.2.2	泄露机制	4
1.2.3	影响范围	5
1.3	项目基本信息	5
1.4	架构概览	6
1.5	本书的分析方法	7
2	技术栈与项目结构	8
2.1	为什么选择 Bun	8
2.1.1	Bun 的关键优势	8
2.1.2	bun:bundle 虚拟模块	8
2.2	TypeScript 配置	9
2.3	构建时宏定义	10
2.4	依赖分析	10
2.4.1	核心依赖	10
2.4.2	特殊依赖: React Canary	11
2.4.3	内部 Stub 包	12
2.5	目录结构详解	12
2.6	设计启示	15
2.6.1	选择合适的运行时	15
2.6.2	虚拟模块实现编译时优化	15
2.6.3	内部包的 Stub 策略	15
3	启动流程与初始化	16
3.1	入口文件分析	16
3.1.1	并行预取: 启动优化的精髓	16
3.1.2	反调试保护	17
3.1.3	配置迁移系统	17

3.2	初始化流程	18
3.2.1	Commander.js 命令注册	18
3.2.2	两种运行模式	19
3.2.3	信任对话框	19
3.3	延迟预取	20
3.4	GrowthBook 功能开关初始化	20
3.5	会话管理	21
3.5.1	会话 ID	21
3.5.2	并发会话检测	21
3.6	遥测与启动日志	21
3.7	设计启示	22
3.7.1	利用模块求值时间	22
3.7.2	分层初始化	22
3.7.3	安全优先	22

II 核心引擎 23

4	核心查询引擎 QueryEngine	24
4.1	QueryEngine 概述	24
4.1.1	设计定位	24
4.1.2	配置接口	25
4.2	submitMessage: 核心方法	26
4.2.1	执行流程	26
4.2.2	权限追踪包装	27
4.2.3	思维模式配置	28
4.3	查询循环 (query.ts)	28
4.3.1	循环状态	28
4.3.2	循环的核心逻辑	29
4.3.3	多层压缩管线	30
4.3.4	max_output_tokens 恢复机制	31
4.4	思维规则	31
4.5	ToolUseContext: 工具执行上下文	32
4.6	Token 预算追踪	33
4.7	设计启示	34
4.7.1	AsyncGenerator 模式	34
4.7.2	多层压缩管线	34
4.7.3	恢复机制	34

5	上下文处理与 System Prompt	35
5.1	上下文的重要性	35
5.2	context.ts 源码分析	35
5.2.1	Git 状态收集	35
5.2.2	系统上下文	37
5.2.3	用户上下文	37
5.2.4	系统提示注入控制	38
5.3	System Prompt 的构建	39
5.3.1	构建流程	39
5.3.2	Prompt 的分层结构	39
5.3.3	Prompt 缓存优化	40
5.4	记忆系统 (Memory)	41
5.4.1	CLAUDE.md 层级	41
5.4.2	持久记忆目录 (memdir)	41
5.4.3	自动记忆提取	41
5.5	附件系统	41
5.6	嵌套记忆路径去重	42
5.7	设计启示	42
5.7.1	分层上下文模型	42
5.7.2	缓存友好的设计	42
5.7.3	快照 vs 实时	43
6	工具系统 (Tool System)	44
6.1	架构概述	44
6.2	工具类型定义	44
6.2.1	核心接口	44
6.2.2	权限上下文	45
6.3	工具注册表 (tools.ts)	45
6.3.1	静态工具注册	45
6.3.2	条件工具注册	46
6.3.3	懒加载 (打破循环依赖)	47
6.3.4	完整工具列表	47
6.4	工具分类详解	48
6.4.1	文件操作工具	48
6.4.2	执行工具	49
6.4.3	网络工具	49
6.4.4	代理与协调工具	49
6.4.5	模式控制工具	49

目录	5
6.4.6 特殊工具	50
6.5 工具过滤机制	50
6.5.1 拒绝规则过滤	50
6.5.2 REPL 模式工具隐藏	50
6.5.3 简单模式	51
6.6 工具池组装	51
6.7 ToolUseContext 深入	52
6.8 设计启示	53
6.8.1 声明式工具定义	53
6.8.2 条件加载与死代码消除	53
6.8.3 工具池的缓存感知排序	53
7 命令系统 (Command System)	55
7.1 概述	55
7.2 命令注册	55
7.3 命令分类	56
7.3.1 Git 操作	56
7.3.2 会话管理	56
7.3.3 配置与状态	56
7.3.4 工具与扩展	57
7.3.5 交互模式	57
7.3.6 认证	57
7.3.7 跨平台	57
7.3.8 Feature Flag 控制的命令	58
7.4 命令与工具的交叉	58
7.5 远程模式命令过滤	58
7.6 命令生命周期	58
7.7 设计启示	59
7.7.1 命令 vs 工具的边界	59
7.7.2 按需加载	59
III 安全与资源管理	60
8 权限与安全模型	61
8.1 概述	61
8.2 权限模式	61
8.2.1 计划模式	61

8.2.2	自动模式与分类器	61
8.3	权限规则	62
8.3.1	规则匹配	62
8.3.2	工具级别过滤	62
8.4	权限检查流程	62
8.5	拒绝追踪	63
8.5.1	本地拒绝追踪	64
8.6	后台代理的权限	64
8.7	危险权限剥离	64
8.8	沙箱系统	64
8.9	Scratchpad 目录	65
8.10	设计启示	65
8.10.1	纵深防御	65
8.10.2	渐进式信任	65
9	上下文压缩与 Token 管理	66
9.1	问题背景	66
9.2	五层压缩管线	66
9.3	第一层：工具结果预算	67
9.4	第二层：Snip 压缩	67
9.4.1	Snip 边界处理	68
9.5	第三层：微压缩	68
9.5.1	缓存微压缩	68
9.6	第四层：上下文折叠	69
9.7	第五层：自动压缩	69
9.7.1	响应式压缩	70
9.8	Token 计数与估算	70
9.9	Token 预算系统	70
9.9.1	任务预算	71
9.10	max_output_tokens 恢复	71
9.11	成本追踪	71
9.11.1	会话成本持久化	72
9.12	设计启示	73
9.12.1	多层防御	73
9.12.2	缓存感知	73
9.12.3	预算管理	73

IV 高级功能 74

10 多智能体协调 (Coordinator Mode) 75

10.1 概述	75
10.2 架构设计	75
10.3 协调器模式的启用	76
10.4 协调器的系统提示	76
10.4.1 核心原则	77
10.5 Worker 工具可用性	77
10.5.1 内部工具隔离	78
10.6 Worker 通知机制	78
10.7 会话模式切换	79
10.8 团队代理 (Agent Swarms)	79
10.9 设计启示	80
10.9.1 提示即架构	80
10.9.2 关注点分离	80

11 MCP 协议与外部集成 81

11.1 什么是 MCP	81
11.2 MCP 在 Claude Code 中的集成	81
11.3 MCP 服务层	81
11.3.1 MCP 服务器连接	82
11.3.2 MCP 配置	82
11.3.3 企业 MCP 管控	82
11.4 MCP 工具与内置工具的融合	83
11.5 MCP 资源	83
11.6 MCP 命令	84
11.7 Elicitation 处理	84
11.8 服务器排除	84
11.9 设计启示	84
11.9.1 开放协议的价值	84
11.9.2 权限统一	85

12 终端 UI: React + Ink 86

12.1 技术选型	86
12.2 为什么用 React 做终端 UI	86
12.3 React Canary 版本	86
12.4 组件库概览	86

12.4.1 核心组件	87
12.4.2 Ink 渲染器封装	87
12.5 JSX 与工具 UI	87
12.6 React Hooks 在 CLI 中的应用	88
12.7 全屏界面 (Screens)	88
12.8 FPS 追踪	88
12.9 输出样式	88
12.10 Vim 模式	89
12.11 Buddy (伴侣精灵)	89
12.12 设计启示	89
12.12.1 React 的通用性	89
12.12.2 组件化的终端 UI	89
12.12.3 性能追踪	89
V 运营与扩展	90
13 Feature Flag 与条件编译	91
13.1 双层 Feature Flag 系统	91
13.2 编译时 Feature Flag (bun:bundle)	91
13.2.1 工作原理	91
13.2.2 开发环境 Polyfill	91
13.2.3 已知 Feature Flag 列表	92
13.3 运行时 Feature Flag (GrowthBook)	93
13.3.1 缓存策略	93
13.3.2 使用场景	93
13.4 环境变量控制	94
13.5 构建时常量	94
13.6 内部 vs 外部构建	94
13.7 设计启示	95
13.7.1 编译时 vs 运行时	95
13.7.2 命名规范	95
14 IDE 桥接与远程会话	96
14.1 桥接系统概述	96
14.2 桥接架构	96
14.2.1 认证	96
14.2.2 消息协议	97

14.2.3 权限回调	97
14.3 远程会话	97
14.3.1 Teleport	97
14.3.2 跨平台传递	98
14.4 服务器模式	98
14.5 LSP 集成	98
14.6 设计启示	98
14.6.1 CLI 优先, IDE 其次	98
15 成本追踪与遥测	100
15.1 成本追踪系统	100
15.1.1 核心数据结构	100
15.1.2 成本计算	100
15.1.3 Fast Mode 追踪	101
15.1.4 格式化输出	101
15.2 会话成本持久化	102
15.3 OpenTelemetry 集成	103
15.3.1 懒加载优化	103
15.4 分析事件	103
15.4.1 安全类型标记	104
15.5 诊断日志	104
15.6 启动性能分析	105
15.7 设计启示	105
15.7.1 隐私优先的遥测	105
15.7.2 多维成本追踪	105
16 插件与技能系统	106
16.1 插件系统	106
16.1.1 架构	106
16.1.2 插件加载	106
16.1.3 插件管理命令	107
16.1.4 版本管理	107
16.1.5 托管插件	107
16.2 技能系统	107
16.2.1 架构	107
16.2.2 内置技能初始化	108
16.2.3 技能工具	108
16.2.4 技能发现	108

16.2.5 技能搜索	108
16.2.6 斜杠命令技能	108
16.3 技能变更检测	109
16.4 遥测	109
16.5 设计启示	109
16.5.1 技能 vs 工具 vs 命令	109
16.5.2 渐进式扩展	110

VI 总结 111

17 设计模式与工程实践总结 112

17.1 架构级模式	112
17.1.1 并行预取 (Parallel Prefetch)	112
17.1.2 分层延迟初始化	112
17.1.3 AsyncGenerator 驱动的流程式处理	112
17.2 代码组织模式	113
17.2.1 条件导入与死代码消除	113
17.2.2 懒加载打破循环依赖	113
17.2.3 Memoize 模式	113
17.2.4 清除缓存的信号	114
17.3 类型系统模式	114
17.3.1 DeepImmutable	114
17.3.2 自文档化类型名	114
17.3.3 品牌类型 (Branded Types)	114
17.4 错误处理模式	115
17.4.1 恢复循环	115
17.4.2 优雅降级	115
17.4.3 分类重试	115
17.5 安全模式	115
17.5.1 纵深防御	115
17.5.2 隐私优先	116
17.5.3 信任边界	116
17.6 性能模式	116
17.6.1 并行化	116
17.6.2 缓存感知排序	116
17.6.3 延迟加载重量级模块	116
17.7 项目规模管理	117

17.8 总结	117
-------------------	-----

Part I

基础架构

Chapter 1

概述与泄露始末

1.1 Claude Code 是什么

Claude Code 是 Anthropic 公司开发的官方 CLI（命令行界面）编程助手。它允许开发者直接在终端中与 Claude 大语言模型交互，执行软件工程任务——包括编辑文件、运行命令、搜索代码库、管理 Git 工作流等。

与 ChatGPT 等网页界面不同，Claude Code 的定位是**嵌入开发者工作流**的原生工具。它不是一个简单的聊天机器人，而是一个拥有完整工具链的 AI 编程代理（Agent）。它可以：

- 读写文件系统中的任意文件
- 执行 Shell 命令（Bash/PowerShell）
- 使用 ripgrep 搜索代码
- 创建 Git 提交和 Pull Request
- 生成子代理（Sub-agent）并行处理任务
- 通过 MCP（Model Context Protocol）连接外部工具
- 在 VS Code 和 JetBrains 等 IDE 中作为后端运行

1.2 泄露事件

1.2.1 时间线

2026 年 3 月 31 日，安全研究员 [Chaofan Shou \(@Friedrice\)](#) 在 X（原 Twitter）上公开了一个发现：

```
> "Claude code source code has been leaked via a map file in their npm registry!"
```

1.2.2 泄露机制

泄露的技术细节如下：

1. **Source Map 文件**：Anthropic 在将 Claude Code 发布到 npm 注册表时，发布包中包含了一个 `.map`（Source Map）文件。

2. **Source Map 中的引用**: 该 Source Map 文件中包含了对完整、未混淆的 TypeScript 源码的引用路径。
3. **R2 存储桶**: 这些源码文件存储在 Anthropic 的 Cloudflare R2 存储桶中, 且可以公开下载。
4. **完整源码可获取**: 通过解析 Source Map 中的路径, 可以下载整个 `src/` 目录的完整源码。

```
1 npm package (.tgz)
2     dist/
3         bundle.js.map ← Source Map 文件
4             sourcesContent 引用 → R2 存储桶中的 TypeScript 源文件
```

1.2.3 影响范围

泄露的源码包含:

- 完整的 `src/` 目录, 约 **1,900 个文件**
- 总计 **512,000+** 行 TypeScript 代码
- 所有核心模块的完整实现

未包含的部分:

- `@ant/*` 开头的 Anthropic 内部专用包 (如 `@ant/computer-use-mcp`、`@ant/claude-for-chrome-mcp`)
- 部分构建脚本和 CI/CD 配置
- 测试文件 (部分)

在公开仓库中, 缺失的内部包被替换为导出空操作 (no-op) 的 stub 模块。

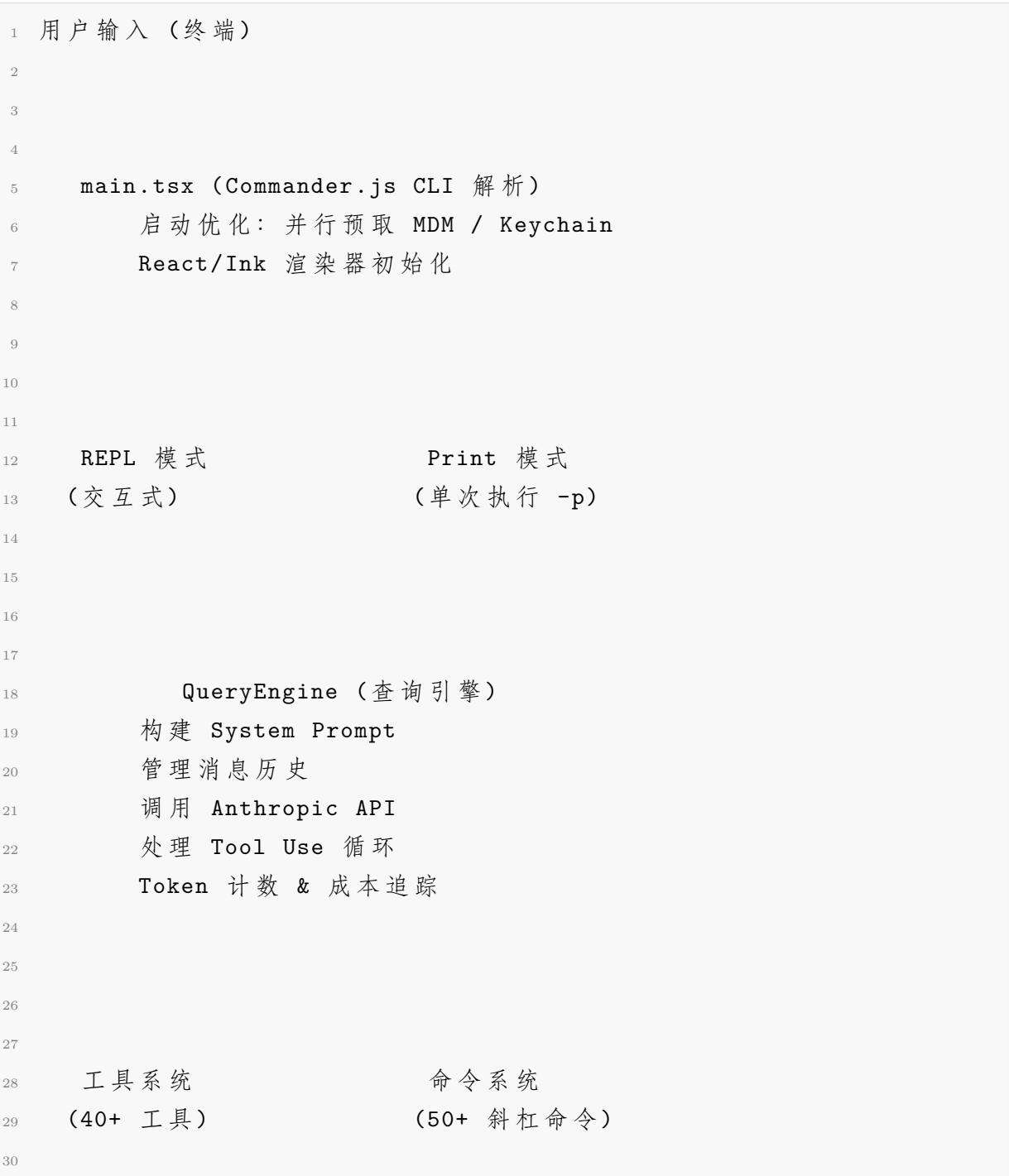
1.3 项目基本信息

属性	值
语言	TypeScript (严格模式)
运行时	Bun v1.3+
终端 UI 框架	React + Ink
CLI 解析	Commander.js (extra-typings)
模式验证	Zod v4
代码搜索	ripgrep
协议支持	MCP SDK、LSP

API 客户端	Anthropic SDK
遥测	OpenTelemetry + gRPC
功能开关	GrowthBook
认证	OAuth 2.0、JWT、macOS Keychain

1.4 架构概览

Claude Code 的整体架构可以用一张图来概括：



```
31
32
33
34         服务层 (Services)
35     Anthropic API 客户端
36     MCP 服务器连接
37     OAuth 认证
38     LSP 语言服务
39     上下文压缩 (Compact)
40     分析 / 遥测
41     插件 / 技能加载
42
```

1.5 本书的分析方法

本书采用自顶向下与自底向上相结合的方法：

1. **自顶向下**：从入口文件 `main.tsx` 开始，沿着启动流程和请求处理链路，逐步深入到每个子系统。
2. **自底向上**：对关键的类型定义（如 `Tool.ts` 中的接口）和工具实现（如 `BashTool`）进行独立的深度分析。
3. **横向对比**：将 Claude Code 的设计决策与业界常见做法（如 LangChain、AutoGPT 等）进行对比，分析其设计取舍。

每章结尾都包含**设计启示**部分，总结该模块中值得借鉴的设计模式和工程实践。

下一章：技术栈与项目结构

Chapter 2

技术栈与项目结构

2.1 为什么选择 Bun

Claude Code 选择 Bun 作为运行时而非 Node.js，这是一个值得深思的技术决策。

2.1.1 Bun 的关键优势

特性	Node.js	Bun	Claude Code 的受益点
启动速度	~40ms	~6ms	CLI 工具对冷启动极度敏感
TypeScript 支持	需要 ts-node/tsx	原生支持	无需编译步骤即可运行.ts/.tsx
内置 bundler	无	有	支持 bun:bundle Feature Flag 系统
包管理器	npm/yarn/pnpm	内置 (bun install)	更快的依赖安装
文件系统操作	标准库	优化的 I/O	大量文件读写操作受益

2.1.2 bun:bundle 虚拟模块

Claude Code 利用 Bun 独有的 bun:bundle 虚拟模块实现编译时功能开关：

```
1 // 生产环境：Bun bundler 在编译时静态替换 feature() 调用
2 import { feature } from 'bun:bundle'
3
4 // 如果 VOICE_MODE 未启用，以下代码会在编译时被完全移除（死代码消除）
5 const voiceCommand = feature('VOICE_MODE')
6   ? require('./commands/voice/index.js').default
7   : null
```

在开发环境中，通过 plugins/bunBundleDev.ts 插件提供运行时 polyfill：

```
1 // plugins/bunBundleDev.ts
2 import { plugin } from 'bun'
3
```

```
4  const enabledFlags = new Set(  
5    (process.env.FEATURE_FLAGS ?? '').split(',').filter(Boolean),  
6  )  
7  
8  plugin({  
9    name: 'bun-bundle-dev',  
10   setup(build) {  
11     build.module('bun:bundle', () => ({  
12       exports: {  
13         feature(flag: string): boolean {  
14           return enabledFlags.has(flag)  
15         },  
16       },  
17       loader: 'object',  
18     })))  
19   },  
20 })
```

开发时通过环境变量启用特定功能：

```
1  FEATURE_FLAGS=KAIROOS,VOICE_MODE bun run src/main.tsx
```

2.2 TypeScript 配置

Claude Code 使用严格模式的 TypeScript：

```
1  {  
2    "compilerOptions": {  
3      "target": "ESNext",  
4      "module": "ESNext",  
5      "moduleResolution": "bundler",  
6      "jsx": "react-jsx",  
7      "strict": true,  
8      "esModuleInterop": true,  
9      "skipLibCheck": true,  
10     "forceConsistentCasingInFileNames": true,  
11     "resolveJsonModule": true,  
12     "baseUrl": ".",  
13     "paths": {
```

```
14     "src/*": ["../src/*"],
15     "bun:bundle": ["../src/types/bun-bundle.d.ts"]
16   },
17   "types": ["bun-types"],
18   "lib": ["ESNext"],
19   "allowImportingTsExtensions": true,
20   "noEmit": true
21 }
22 }
```

关键配置点：

1. `moduleResolution: "bundler"`: 使用 Bun 的模块解析策略。
2. `jsx: "react-jsx"`: 支持 JSX 语法，用于 Ink 终端 UI 组件。
3. 路径别名: `src/*` 映射允许使用绝对导入路径（如 `src/bootstrap/state.js`）。
4. `bun:bundle` 类型映射：将虚拟模块映射到本地类型定义文件。

2.3 构建时宏定义

通过 `bunfig.toml` 定义构建时注入的全局常量：

```
1 [run]
2 preload = ["../plugins/bunBundleDev.ts"]
3
4 [define]
5 "MACRO.VERSION" = '"1.0.0-dev"'
6 "MACRO.BUILD_TIME" = '""'
7 "MACRO.PACKAGE_URL" = '"@anthropic-ai/claude-code"'
8 "MACRO.NATIVE_PACKAGE_URL" = 'undefined'
9 "MACRO.FEEDBACK_CHANNEL" = '"https://github.com/anthropics/claude-
   code/issues"'
10 "MACRO.ISSUES_EXPLAINER" = '"file an issue at ..."'
11 "MACRO.VERSION_CHANGELOG" = '""'
```

这些 `MACRO.*` 常量在代码中直接作为全局变量使用，Bun 在构建/运行时自动替换。

2.4 依赖分析

2.4.1 核心依赖

分类	包名	用途
AI/API	@anthropic-ai/sdk	Anthropic API 客户端
	@anthropic-ai/claude-agent-sdk	Agent SDK
	@aws-sdk/client-bedrock-runtime	AWS Bedrock 支持
	google-auth-library	Google Cloud 认证
终端 UI	react (canary)	React 19 Canary 版本
	react-reconciler (canary)	自定义 React 渲染器
	chalk	终端着色
	cli-boxes	终端边框
	figures	Unicode 符号
CLI	commander	命令解析
	@commander-js/extra-typings	Commander 增强类型
协议	@modelcontextprotocol/sdk	MCP 协议实现
	vscode-jsonrpc	JSON-RPC 通信
	vscode-languageserver-protocol	LSP 协议
数据处理	zod	运行时模式验证
	ajv	JSON Schema 验证
	diff	文本差异比较
	marked	Markdown 解析
工具	execa	子进程管理
	chokidar	文件监听
	fuse.js	模糊搜索
	lodash-es	实用函数库
网络	undici	HTTP 客户端
	ws	WebSocket
	https-proxy-agent	代理支持
安全	xss	XSS 过滤
	proper-lockfile	文件锁
遥测	@opentelemetry/*	OpenTelemetry 全家桶
	@growthbook/growthbook	功能开关

2.4.2 特殊依赖：React Canary

值得注意的是，Claude Code 使用的是 React **19.3.0-canary** 版本，而非稳定版。这是因为 Ink（终端 UI 框架）需要与 `react-reconciler` 的 canary 版本匹配。

2.4.3 内部 Stub 包

被替换的 Anthropic 内部包（使用 `file:stubs/...` 引用）：

```

1 {
2   "@ant/claude-for-chrome-mcp": "file:stubs/@ant/claude-for-chrome-
   -mcp",
3   "@ant/computer-use-input": "file:stubs/@ant/computer-use-input",
4   "@ant/computer-use-mcp": "file:stubs/@ant/computer-use-mcp",
5   "@ant/computer-use-swift": "file:stubs/@ant/computer-use-swift",
6   "@anthropic-ai/mcpb": "file:stubs/@anthropic-ai/mcpb",
7   "@anthropic-ai/sandbox-runtime": "file:stubs/anthropic-ai/
   sandbox-runtime"
8 }
```

这些包在内部用于：

- **Chrome 浏览器控制** (`claude-for-chrome-mcp`)
- **计算机使用** (`computer-use-*`)：屏幕截图、鼠标/键盘操控
- **沙箱运行时** (`sandbox-runtime`)：安全隔离执行环境

2.5 目录结构详解

```

1 src/
2   main.tsx                # 入口文件：Commander.js CLI 解析 +
   React/Ink 初始化
3   commands.ts             # 命令注册表：管理所有斜杠命令
4   tools.ts                # 工具注册表：管理所有 AI 可调用工具
5   Tool.ts                 # 工具类型定义：接口、权限模型、进度
   类型
6   QueryEngine.ts          # 核心查询引擎：LLM API 调用、流式响
   应、工具循环
7   query.ts                # 查询循环：消息处理、自动压缩、Token
   管理
8   context.ts              # 上下文收集：System/User Context 生
   成
9   cost-tracker.ts         # 成本追踪：Token 计数、费用计算
10
11  commands/                # 斜杠命令实现（~50 个命令）
12    commit.ts              #   /commit - Git 提交
```

```

13      review.ts          # /review - 代码审查
14      compact/           # /compact - 上下文压缩
15      config/            # /config - 设置管理
16      doctor/            # /doctor - 环境诊断
17      memory/            # /memory - 持久记忆管理
18      mcp/               # /mcp - MCP 服务器管理
19      skills/            # /skills - 技能管理
20      tasks/             # /tasks - 任务管理
21      vim/               # /vim - Vim 模式
22      ...
23
24  tools/                 # 工具实现 (~40 个工具)
25      AgentTool/         # 子代理生成
26      BashTool/          # Shell 命令执行
27      FileReadTool/      # 文件读取
28      FileWriteTool/     # 文件创建/覆盖
29      FileEditTool/      # 文件部分编辑
30      GlobTool/          # 文件模式匹配
31      GrepTool/          # ripgrep 内容搜索
32      WebFetchTool/      # URL 内容获取
33      WebSearchTool/     # 网页搜索
34      SkillTool/         # 技能执行
35      MCPTool/           # MCP 工具调用
36      LSPTool/           # 语言服务器协议
37      TeamCreateTool/    # 团队代理创建
38      SendMessageTool/   # 代理间消息
39      EnterPlanModeTool/ # 进入计划模式
40      EnterWorktreeTool/ # Git Worktree 隔离
41      ...
42
43  components/            # Ink UI 组件 (~140 个)
44      REPL.tsx           # 主 REPL 界面
45      MessageSelector.tsx # 消息选择器
46      AgentProgressLine.tsx # 代理进度显示
47      Spinner.tsx        # 加载动画
48      ...
49
50  hooks/                 # React Hooks
51      toolPermission/     # 工具权限检查
52      fileSuggestions.ts  # 文件建议

```

```

53         useCanUseTool.ts      # 工具使用权限
54         ...
55
56     services/                  # 外部服务集成
57         api/                   # Anthropic API 客户端
58         mcp/                   # MCP 服务器管理
59         oauth/                 # OAuth 认证
60         lsp/                   # LSP 管理器
61         compact/              # 上下文压缩服务
62         analytics/            # GrowthBook 分析
63         plugins/              # 插件加载
64         policyLimits/         # 策略限制
65         extractMemories/      # 自动记忆提取
66         ...
67
68     bridge/                    # IDE 集成桥
69         bridgeMain.ts         # 桥主循环
70         bridgeMessaging.ts    # 消息协议
71         ...
72
73     coordinator/              # 多代理协调器
74         coordinatorMode.ts    # 协调器模式
75
76     state/                    # 状态管理
77         AppStateStore.tsx     # 全局应用状态
78         store.ts              # 状态存储
79         onChangeAppState.ts   # 状态变更监听
80
81     plugins/                  # 插件系统
82     skills/                   # 技能系统
83     memdir/                   # 持久记忆目录
84     tasks/                    # 任务管理
85     voice/                    # 语音输入
86     vim/                      # Vim 模式
87     remote/                   # 远程会话
88     server/                   # 服务器模式
89     schemas/                  # Zod 配置模式
90     migrations/               # 配置迁移
91     entrypoints/              # 入口初始化逻辑
92     types/                    # 类型定义

```

```
93     utils/                # 工具函数
94     constants/           # 常量定义
95     ink/                 # Ink 渲染器封装
96     buddy/              # 伴侣精灵 (彩蛋)
97     upstreamproxy/       # 代理配置
```

2.6 设计启示

2.6.1 选择合适的运行时

Claude Code 选择 Bun 而非 Node.js 的决策展示了一个重要原则：**CLI 工具应该为冷启动时间进行优化**。Bun 的 6ms 启动时间 vs Node.js 的 40ms，对于频繁调用的 CLI 工具来说影响显著。

2.6.2 虚拟模块实现编译时优化

`bun:bundle` 的 Feature Flag 系统展示了一种优雅的**死代码消除**方案。未启用的功能在编译时被完全移除，减小了最终产物的体积，同时避免了运行时检查的开销。

2.6.3 内部包的 Stub 策略

对于无法公开的内部依赖，使用 `file:stubs/` 替换为空实现是一种实用的解耦策略，使得核心代码可以在缺失内部依赖的情况下独立运行。

下一章：[启动流程与初始化](#)

Chapter 3

启动流程与初始化

3.1 入口文件分析

Claude Code 的入口文件是 `src/main.tsx`。这是整个应用中最大的文件之一，包含了从 CLI 参数解析到 REPL 启动的完整初始化链路。

3.1.1 并行预取：启动优化的精髓

`main.tsx` 的前几行代码展现了 Claude Code 对启动性能的极致追求：

```
1 // 这些副作用必须在所有其他 import 之前运行：
2 // 1. profileCheckpoint 在重量级模块求值开始前标记入口
3 // 2. startMdmRawRead 启动 MDM 子进程 (plutil/reg query)，使其与
4 //    后续 ~135ms 的 import 并行运行
5 // 3. startKeychainPrefetch 并行启动两个 macOS 钥匙串读取
6 //    (OAuth + 遗留 API key)
7
8 import { profileCheckpoint } from './utils/startupProfiler.js';
9 profileCheckpoint('main_tsx_entry');
10
11 import { startMdmRawRead } from './utils/settings/mdm/rawRead.js';
12 startMdmRawRead();
13
14 import { startKeychainPrefetch } from './utils/secureStorage/
    keychainPrefetch.js';
15 startKeychainPrefetch();
```

关键洞察：JavaScript 模块的 `import` 语句在执行时会触发模块求值 (module evaluation)，这个过程需要约 135ms。Claude Code 利用这段“免费”的时间，将 I/O 密集型操作（如读取系统设置和钥匙串）提前启动，使其与模块加载并行执行。

```
1 时间线：
2 0ms      profileCheckpoint('main_tsx_entry')
```

```

3      startMdmRawRead()          ← 启动 MDM 子进程 (非阻塞)
4      startKeychainPrefetch()   ← 启动 钥匙串读取 (非阻塞)
5
6
7      模块求值 (~135ms)
8      - import chalk, React, Commander...
9      - import services, tools, commands...
10
11
12
13      MDM 读取 & 钥匙串读取 并行进行
14      (在模块求值期间已完成)
15
16
17 135ms      profileCheckpoint('main_tsx_imports_loaded')
18          ← 此时 MDM 和 Keychain 数据通常已就绪

```

3.1.2 反调试保护

在启动阶段还有一个安全措施——反调试检测：

```

1  // 如果非内部用户使用调试/检查模式运行，直接退出
2  if ("external" !== 'ant' && isBeingDebugged()) {
3      process.exit(1);
4  }

```

isBeingDebugged() 检查三个信号：

1. process.execArgv 中的 --inspect / --debug 标志
2. NODE_OPTIONS 环境变量中的检查标志
3. Node.js inspector 模块是否激活

注意 "external" !== 'ant' 这个条件：在内部构建中，字符串 "external" 会被替换为 "ant"，使得内部用户可以自由调试。

3.1.3 配置迁移系统

```

1  const CURRENT_MIGRATION_VERSION = 11;
2
3  function runMigrations(): void {

```

```
4  if (getGlobalConfig().migrationVersion !==
    CURRENT_MIGRATION_VERSION) {
5      migrateAutoUpdatesToSettings();
6      migrateBypassPermissionsAcceptedToSettings();
7      migrateEnableAllProjectMcpServersToSettings();
8      resetProToOpusDefault();
9      migrateSonnet1mToSonnet45();
10     migrateLegacyOpusToCurrent();
11     migrateSonnet45ToSonnet46();
12     migrateOpusToOpus1m();
13     migrateReplBridgeEnabledToRemoteControlAtStartup();
14     // ...
15     saveGlobalConfig(prev => ({
16         ...prev,
17         migrationVersion: CURRENT_MIGRATION_VERSION
18     }));
19 }
20 }
```

这个迁移系统的设计特点：

- **版本号控制**：只有当配置版本号不匹配时才执行迁移
- **幂等性**：每个迁移函数自身处理“已迁移”状态
- **模型演进追踪**：可以清晰看到 Anthropic 的模型命名变化历史（Sonnet 1M → Sonnet 4.5 → Sonnet 4.6、Opus → Opus 1M 等）

3.2 初始化流程

3.2.1 Commander.js 命令注册

main.tsx 使用 Commander.js 定义了丰富的 CLI 选项：

```
1  const program = new CommanderCommand()
2      .name('claude')
3      .description('Claude Code CLI')
4      .option('-p, --print <prompt>', '单次执行模式（非交互式）')
5      .option('--model <model>', '指定模型')
6      .option('--permission-mode <mode>', '权限模式')
7      .option('--tools <preset>', '工具预设')
8      .option('--add-dir <path...>', '添加额外工作目录')
```

```
9  .option('--bare', '精简模式')
10 .option('--verbose', '详细输出')
11 // ... 更多选项
```

3.2.2 两种运行模式

Claude Code 有两种主要运行模式：

Print 模式 (-p) — 非交互式，单次执行：

```
1  claude -p "请解释这个函数的作用"
```

REPL 模式 — 交互式，持续对话：

```
1  claude # 直接启动交互式会话
```

3.2.3 信任对话框

在交互模式下，首次使用时会显示信任对话框：

```
1  function prefetchSystemContextIfSafe(): void {
2      const isNonInteractiveSession = getIsNonInteractiveSession();
3
4      // 非交互模式下，信任是隐式的
5      if (isNonInteractiveSession) {
6          void getSystemContext();
7          return;
8      }
9
10     // 交互模式下，只在信任已建立时才预取
11     const hasTrust = checkHasTrustDialogAccepted();
12     if (hasTrust) {
13         void getSystemContext();
14     }
15     // 否则等待信任建立后再获取
16 }
```

安全考量：getSystemContext() 会执行 git status 等 Git 命令。Git 命令可以通过 hooks 和配置（如 core.fsmonitor、diff.external）执行任意代码。因此，在用户确认信任当前目录之前，不会执行任何 Git 命令。

3.3 延迟预取

Claude Code 将非关键的预取操作延迟到首次渲染之后：

```
1 export function startDeferredPrefetches(): void {
2     // 如果只是测量启动性能，跳过所有预取
3     if (isEnvTruthy(process.env.CLAUDE_CODE_EXIT_AFTER_FIRST_RENDER)
4         ) {
5         return;
6     }
7
8     // --bare 模式：跳过所有预取
9     if (isBareMode()) {
10        return;
11    }
12
13    // 这些在首次渲染之后运行，不阻塞初始画面：
14    // - initUser: 用户信息初始化
15    // - getUserContext: CLAUDE.md 读取
16    // - tips: 使用提示
17    // - countFiles: 文件计数
18    // - modelCapabilities: 模型能力查询
19    // - change detectors: 设置/技能变更检测
20 }
```

设计原则：将用户感知到的启动时间与实际初始化时间分离。用户看到界面（首次渲染）后，后台继续完成剩余初始化。

3.4 GrowthBook 功能开关初始化

```
1 import { initializeGrowthBook } from './services/analytics/
   growthbook.js';
```

GrowthBook 是 Claude Code 使用的功能开关（Feature Flag）服务。它在启动时初始化，用于：

- 控制 A/B 测试
- 灰度发布新功能
- 远程配置管理

与 `bun:bundle` 的编译时 Feature Flag 不同, GrowthBook 提供的是**运行时功能开关**, 可以在不更新版本的情况下远程开关功能。

3.5 会话管理

3.5.1 会话 ID

每次启动都会生成一个唯一的会话 ID:

```
1 const sessionId = getSessionId(); // UUID v4
```

会话 ID 用于:

- 关联遥测数据
- 持久化会话记录
- 恢复 (Resume) 会话

3.5.2 并发会话检测

```
1 import { countConcurrentSessions, registerSession } from 'src/
  utils/concurrentSessions.js';
```

Claude Code 支持多个实例同时运行, 并通过文件锁机制追踪并发会话数量。

3.6 遥测与启动日志

```
1 async function logStartupTelemetry(): Promise<void> {
2   if (isAnalyticsDisabled()) return;
3
4   const [isGit, worktreeCount, ghAuthStatus] = await Promise.all([
5     getIsGit(),
6     getWorktreeCount(),
7     getGhAuthStatus()
8   ]);
9
10  logEvent('tengu_startup_telemetry', {
11    is_git: isGit,
12    worktree_count: worktreeCount,
13    gh_auth_status: ghAuthStatus,
```

```
14     sandbox_enabled: SandboxManager.isSandboxingEnabled(),
15     auto_updater_disabled: isAutoUpdaterDisabled(),
16     // ...
17   });
18 }
```

注意内部代号 **”tengu”**（天狗）——这是 Claude Code 的内部项目名。所有遥测事件都以 `tengu_` 为前缀。

3.7 设计启示

3.7.1 利用模块求值时间

在 JavaScript/TypeScript 应用中，模块加载是一段“不可避免”的开销。Claude Code 展示了如何将这段时间变废为宝：在模块求值开始前启动异步 I/O 操作，让它们与模块加载并行进行。

3.7.2 分层初始化

将初始化分为多个阶段：

1. **最早**：性能关键的异步 I/O（MDM、Keychain）
2. **模块加载期间**：自动完成（JavaScript 引擎的工作）
3. **首次渲染前**：必要的同步初始化
4. **首次渲染后**：延迟预取（不影响用户感知）

3.7.3 安全优先

在信任建立之前不执行可能危险的操作（如 Git 命令），体现了安全与性能之间的平衡。

下一章：[核心查询引擎 QueryEngine](#)

Part II

核心引擎

Chapter 4

核心查询引擎 QueryEngine

4.1 QueryEngine 概述

QueryEngine 是 Claude Code 的核心类，定义在 `src/QueryEngine.ts` 中（约 46,000 行）。它拥有整个对话生命周期的状态，管理与 Anthropic API 的通信、工具调用循环、流式响应处理等。

4.1.1 设计定位

```
1  /**
2   * QueryEngine 拥有查询生命周期和会话状态。
3   * 它将核心逻辑从 ask() 提取为独立类，既可用于 headless/SDK 路径，
4   * 也可（在未来阶段）用于 REPL。
5   *
6   * 每个对话一个 QueryEngine。每次 submitMessage() 调用在同一对话中
7   * 启动一个新的回合 (turn)。状态（消息、文件缓存、使用量等）
8   * 在回合之间持久存在。
9   */
10 export class QueryEngine {
11     private config: QueryEngineConfig
12     private mutableMessages: Message[]
13     private abortController: AbortController
14     private permissionDenials: SDKPermissionDenial[]
15     private totalUsage: NonNullableUsage
16     private hasHandledOrphanedPermission = false
17     private readFileState: FileStateCache
18     private discoveredSkillNames = new Set<string>()
19     private loadedNestedMemoryPaths = new Set<string>()
20
21     constructor(config: QueryEngineConfig) {
22         this.config = config
23         this.mutableMessages = config.initialMessages ?? []
```

```

24     this.abortController = config.abortController ??
        createAbortController()
25     this.permissionDenials = []
26     this.readFileState = config.readFileCache
27     this.totalUsage = EMPTY_USAGE
28 }
29 }

```

4.1.2 配置接口

```

1  export type QueryEngineConfig = {
2      cwd: string // 工作目录
3      tools: Tools // 可用工具列表
4      commands: Command[] // 可用命令列表
5      mcpClients: MCPServerConnection[] // MCP 服务器连接
6      agents: AgentDefinition[] // 代理定义
7      canUseTool: CanUseToolFn // 工具权限检查函数
8      getAppState: () => AppState // 全局状态获取
9      setAppState: (f) => void // 全局状态更新
10     initialMessages?: Message[] // 初始消息历史
11     readFileCache: FileStateCache // 文件状态缓存
12     customSystemPrompt?: string // 自定义系统提示
13     appendSystemPrompt?: string // 追加系统提示
14     userSpecifiedModel?: string // 用户指定模型
15     fallbackModel?: string // 备用模型
16     thinkingConfig?: ThinkingConfig // 思维模式配置
17     maxTurns?: number // 最大回合数
18     maxBudgetUsd?: number // 最大预算 (美元)
19     taskBudget?: { total: number } // 任务预算
20     jsonSchema?: Record<string, unknown> // 结构化输出 Schema
21     verbose?: boolean // 详细模式
22     replayUserMessages?: boolean // 重放用户消息
23     handleElicitation?: (...) => Promise // URL 引出处理
24     includePartialMessages?: boolean // 包含部分消息
25     setSDKStatus?: (status) => void // SDK 状态回调
26     abortController?: AbortController // 中止控制器
27     orphanedPermission?: OrphanedPermission // 孤立权限
28     snipReplay?: (...) => ... | undefined // Snip 重放处理器
29 }

```

4.2 submitMessage: 核心方法

submitMessage 是一个 AsyncGenerator——它通过 yield 逐步产出流式事件，使调用方能实时获取 API 响应的每一部分。

```
1  async *submitMessage(  
2    prompt: string | ContentBlockParam[],  
3    options?: { uuid?: string; isMeta?: boolean },  
4  ): AsyncGenerator<SDKMessage, void, unknown> {
```

4.2.1 执行流程

```
1  submitMessage(prompt)  
2  
3  
4  1. 清理状态 & 设置工作目录  
5  
6  
7  2. 包装 canUseTool (追踪权限拒绝)  
8  
9  
10 3. 解析模型 & 思维配置  
11  
12  
13 4. 获取 System Prompt  
14     fetchSystemPromptParts()  
15     默认系统提示 (defaultSystemPrompt)  
16     用户上下文 (baseUserContext)  
17     系统上下文 (systemContext)  
18  
19  
20 5. 构建 ProcessUserInputContext  
21  
22  
23 6. 处理用户输入 → processUserInput()  
24     斜杠命令检测
```

```
25     附件处理
26     生成用户消息
27
28
29 7. 调用查询循环 → query()
30     构建完整 System Prompt
31     调用 Anthropic API (流式)
32     处理 Tool Use 请求
33     执行工具 → 返回结果
34     自动压缩 (如果需要)
35     循环直到模型停止
36
37
38 8. 产出最终状态 & 使用量统计
```

4.2.2 权限追踪包装

```
1  const wrappedCanUseTool: CanUseToolFn = async (
2    tool, input, toolUseContext, assistantMessage, toolUseID,
3    forceDecision,
4  ) => {
5    const result = await canUseTool(
6      tool, input, toolUseContext, assistantMessage, toolUseID,
7      forceDecision,
8    )
9
10   // 追踪拒绝以供 SDK 报告
11   if (result.behavior !== 'allow') {
12     this.permissionDenials.push({
13       tool_name: sdkCompatToolName(tool.name),
14       tool_use_id: toolUseID,
15       tool_input: input,
16     })
17   }
18
19   return result
20 }
```

每次工具调用的权限决定都被包装和追踪。被拒绝的工具调用会被记录，最终通过 SDK

事件报告给调用方。

4.2.3 思维模式配置

```
1  const initialThinkingConfig: ThinkingConfig = thinkingConfig
2    ? thinkingConfig
3    : shouldEnableThinkingByDefault() !== false
4    ? { type: 'adaptive' } // 自适应思维
5    : { type: 'disabled' } // 禁用思维
```

Claude Code 支持三种思维模式：

- **adaptive**: 模型自行决定是否深度思考
- **enabled**: 始终启用扩展思考
- **disabled**: 禁用扩展思考

4.3 查询循环 (query.ts)

query.ts 实现了与 API 交互的核心循环——这是 Claude Code 最复杂的控制流之一。

4.3.1 循环状态

```
1  type State = {
2    messages: Message[] // 消息历史
3    toolUseContext: ToolUseContext // 工具使用上下文
4    autoCompactTracking: AutoCompactTrackingState // 自动压缩追踪
5    maxOutputTokensRecoveryCount: number // 输出 Token 恢复计数
6    hasAttemptedReactiveCompact: boolean // 是否尝试过响应式压缩
7    maxOutputTokensOverride: number | undefined // 输出 Token 覆盖值
8    pendingToolUseSummary: Promise<...> | undefined // 待处理工具使用摘要
9    stopHookActive: boolean | undefined // 停止钩子是否激活
10   turnCount: number // 回合计数
11   transition: Continue | undefined // 上一次迭代的继续原因
```

```
12 }
```

4.3.2 循环的核心逻辑

```
1  export async function* query(params: QueryParams): AsyncGenerator
    <...> {
2      const consumedCommandUids: string[] = []
3      const terminal = yield* queryLoop(params, consumedCommandUids)
4
5      for (const uuid of consumedCommandUids) {
6          notifyCommandLifecycle(uuid, 'completed')
7      }
8      return terminal
9  }
10
11 async function* queryLoop(params, consumedCommandUids) {
12     // ... 初始化状态 ...
13
14     while (true) {
15         // 1. 技能发现预取
16         const pendingSkillPrefetch = skillPrefetch?.
            startSkillDiscoveryPrefetch(...)
17
18         // 2. 发出请求开始事件
19         yield { type: 'stream_request_start' }
20
21         // 3. 获取压缩边界后的消息
22         let messagesForQuery = [...getMessagesAfterCompactBoundary(
            messages)]
23
24         // 4. 应用工具结果预算
25         messagesForQuery = await applyToolResultBudget(
            messagesForQuery, ...)
26
27         // 5. Snip 压缩 (如果启用)
28         if (feature('HISTORY_SNIP')) {
29             const snipResult = snipModule!.snipCompactIfNeeded(
                messagesForQuery)
30             messagesForQuery = snipResult.messages
```

```

31     }
32
33     // 6. 微压缩
34     const microcompactResult = await deps.microcompact(
35         messagesForQuery, ...)
36     messagesForQuery = microcompactResult.messages
37
38     // 7. 上下文折叠 (如果启用)
39     if (feature('CONTEXT_COLLAPSE') && contextCollapse) {
40         const collapseResult = await contextCollapse.
41             applyCollapsesIfNeeded(...)
42         messagesForQuery = collapseResult.messages
43     }
44
45     // 8. 构建完整系统提示
46     const fullSystemPrompt = asSystemPrompt(
47         appendSystemContext(systemPrompt, systemContext)
48     )
49
50     // 9. 调用 API 并处理响应
51     // ... 流式处理 + 工具调用 ...
52
53     // 10. 判断是否继续循环
54     // - 如果有工具调用 → 继续
55     // - 如果模型结束 → 退出
56     // - 如果达到 max_output_tokens → 恢复尝试
57 }
58 }

```

4.3.3 多层压缩管线

查询循环中的消息在发送到 API 之前，会经过一个**多层压缩管线**：

```

1  原始消息历史
2
3
4  工具结果预算 (applyToolResultBudget)
5      → 替换超大工具结果为摘要
6

```

```

7   Snip 压缩 (snipCompactIfNeeded)
8       → 移除旧的对话片段
9
10  微压缩 (microcompact)
11       → 压缩单个工具输出的冗余内容
12
13  上下文折叠 (applyCollapsesIfNeeded)
14       → 将多轮交互折叠为摘要
15
16  自动压缩 (autoCompact)
17       → 当接近上下文窗口限制时触发
18
19  发送到 API

```

4.3.4 max_output_tokens 恢复机制

当模型输出被截断时，Claude Code 会自动尝试恢复：

```

1  const MAX_OUTPUT_TOKENS_RECOVERY_LIMIT = 3
2
3  function isWithheldMaxOutputTokens(msg: Message | undefined):
4      boolean {
5      return msg?.type === 'assistant' && msg.apiError === '
6      max_output_tokens'
7  }

```

最多尝试 3 次恢复。在恢复过程中，SDK 调用方不会收到错误消息——错误被”扣留” (withheld)，直到确认恢复循环是否成功。

4.4 思维规则

源码中包含一段幽默的注释，描述了处理思维块 (thinking blocks) 的规则：

```

1  /**
2   * 思维规则冗长而繁琐。它们需要大量的长时间思考和深度冥想，
3   * 才能让一个巫师将其理解透彻。
4   *
5   * 规则如下：
6   * 1. 包含 thinking 或 redacted_thinking 块的消息必须属于

```



```

7  *      max_thinking_length > 0 的查询
8  * 2. thinking 块不能是块中的最后一条消息
9  * 3. 思维块必须在助手轨迹期间保持完整
10 *      (单个回合, 或如果该回合包含 tool_use 块,
11 *      则也包括其后续的 tool_result 和跟随的助手消息)
12 *
13 * 好好记住这些规则, 年轻的巫师。因为它们是思维的规则,
14 * 而思维的规则就是宇宙的规则。如果你不遵守这些规则,
15 * 你将受到一整天调试和拔头发的惩罚。
16 */

```

这段注释反映了 Anthropic 工程师在调试思维块处理逻辑时遇到的痛苦经验。

4.5 ToolUseContext: 工具执行上下文

`ToolUseContext` 是传递给每个工具调用的丰富上下文对象:

```

1  export type ToolUseContext = {
2    options: {
3      commands: Command[]
4      debug: boolean
5      mainLoopModel: string
6      tools: Tools
7      verbose: boolean
8      thinkingConfig: ThinkingConfig
9      mcpClients: MCPServerConnection[]
10     mcpResources: Record<string, ServerResource[]>
11     isNonInteractiveSession: boolean
12     agentDefinitions: AgentDefinitionsResult
13     maxBudgetUsd?: number
14     customSystemPrompt?: string
15     appendSystemPrompt?: string
16     refreshTools?: () => Tools
17   }
18   abortController: AbortController
19   readFileState: FileStateCache
20   getAppState(): AppState
21   setAppState(f: (prev: AppState) => AppState): void
22   setAppStateForTasks?: (...) => void

```

```

23  handleElicitation?: (...) => Promise<ElicitResult>
24  setToolJSX?: SetToolJSXFn
25  addNotification?: (notif: Notification) => void
26  appendSystemMessage?: (msg) => void
27  sendOSNotification?: (opts) => void
28  messages: Message[]
29  fileReadingLimits?: { maxTokens?: number; maxSizeBytes?: number
    }
30  globLimits?: { maxResults?: number }
31  toolDecisions?: Map<string, { source; decision; timestamp }>
32  queryTracking?: QueryChainTracking
33  contentReplacementState?: ContentReplacementState
34  renderedSystemPrompt?: SystemPrompt
35  // ... 更多字段
36 }

```

关键设计点：

1. **readFileState**: 文件状态 LRU 缓存。避免重复读取同一文件。
2. **setAppStateForTasks**: 允许嵌套子代理更新根状态存储, 即使子代理的 **setAppState** 是空操作。
3. **contentReplacementState**: 工具结果预算的替换状态, 跨轮次持久化。
4. **renderedSystemPrompt**: 父代理的渲染系统提示的冻结快照, 用于 fork 子代理共享缓存。

4.6 Token 预算追踪

```

1  const budgetTracker = feature('TOKEN_BUDGET')
2    ? createBudgetTracker()
3    : null

```

当 **TOKEN_BUDGET** Feature Flag 启用时, 查询循环会追踪 Token 消耗并与预算比较。

任务预算 (Task Budget): 通过 API 的 `output_config.task_budget` 参数设置整个代理回合的预算。每次压缩后需要重新计算剩余预算。

4.7 设计启示

4.7.1 AsyncGenerator 模式

使用 `async function*` 实现流式数据处理是一个优雅的设计。调用方可以通过 `for await...of` 消费流，也可以使用 `.next()` 精确控制步进。

4.7.2 多层压缩管线

五层压缩看似复杂，但每层解决的问题不同：

- **工具结果预算**：控制单个工具输出的大小
- **Snip**：移除老旧的历史
- **微压缩**：局部优化
- **上下文折叠**：语义级别的压缩
- **自动压缩**：全局兜底

4.7.3 恢复机制

对 `max_output_tokens` 的自动恢复展示了一个重要原则：**AI 应用应该对 API 的边界条件有鲁棒的处理策略**，而不是简单地向用户抛出错误。

下一章：[上下文处理与 System Prompt](#)

Chapter 5

上下文处理与 System Prompt

5.1 上下文的重要性

在 LLM 应用中，上下文（Context）的构建质量直接决定了模型的表现。Claude Code 对上下文处理的设计展现了工业级 AI 应用的精细程度。

上下文分为两个层次：

1. **系统上下文 (System Context)**: 关于运行环境的信息（Git 状态、日期等）
2. **用户上下文 (User Context)**: 来自 CLAUDE.md 配置文件的项目特定指令

5.2 context.ts 源码分析

5.2.1 Git 状态收集

```
1 export const getGitStatus = memoize(async (): Promise<string |  
    null> => {  
2     const isGit = await getIsGit()  
3     if (!isGit) return null  
4  
5     try {  
6         // 并行执行 5 个 Git 命令  
7         const [branch, mainBranch, status, log, userName] = await  
            Promise.all([  
8             getBranch(),  
9             getDefaultBranch(),  
10            execFileNoThrow(gitExe(), ['--no-optional-locks', 'status',  
                '--short'], {  
11                preserveOutputOnError: false,  
12            }).then(({ stdout }) => stdout.trim()),  
13            execFileNoThrow(gitExe(), ['--no-optional-locks', 'log', '--  
                oneline', '-n', '5'], {  
14                preserveOutputOnError: false,
```

```

15     }).then(({ stdout }) => stdout.trim()),
16     execFileNoThrow(gitExe(), ['config', 'user.name'], {
17         preserveOutputOnError: false,
18     }).then(({ stdout }) => stdout.trim()),
19 ])
20
21 // 状态超过 2000 字符时截断
22 const truncatedStatus = status.length > MAX_STATUS_CHARS
23   ? status.substring(0, MAX_STATUS_CHARS) +
24     '\n... (truncated because it exceeds 2k characters. If you
25       need more information, run "git status" using BashTool
26       )'
27   : status
28
29 return [
30     `This is the git status at the start of the conversation...`,
31     ,
32     `Current branch: ${branch}`,
33     `Main branch (you will usually use this for PRs): ${
34       mainBranch}`,
35     ...(userName ? [`Git user: ${userName}`] : []),
36     `Status:\n${truncatedStatus || '(clean)'} `,
37     `Recent commits:\n${log}`,
38   ].join('\n\n')
39 } catch (error) {
40   logError(error)
41   return null
42 }
43 })

```

关键设计点：

1. **Promise.all 并行执行**：5 个 Git 命令同时运行，而非串行等待。
2. **--no-optional-locks**：避免与其他 Git 进程产生锁冲突。
3. **memoize**：结果缓存，整个对话期间只执行一次。
4. **截断保护**：Git status 超过 2000 字符时截断，并提示模型使用 BashTool 获取完整信息。
5. **快照语义**：“这是对话开始时的 git status。注意这个状态是时间快照，不会在对话期间更新。”

5.2.2 系统上下文

```

1 export const getSystemContext = memoize(
2   async (): Promise<{ [k: string]: string }> => {
3     // 在远程模式下或禁用 Git 指令时跳过 Git 状态
4     const gitStatus =
5       isEnvTruthy(process.env.CLAUDE_CODE_REMOTE) ||
6       !shouldIncludeGitInstructions()
7       ? null
8       : await getGitStatus()
9
10    // 缓存破坏注入 (仅 Anthropic 内部使用)
11    const injection = feature('BREAK_CACHE_COMMAND')
12      ? getSystemPromptInjection()
13      : null
14
15    return {
16      ...(gitStatus && { gitStatus }),
17      ...(injection ? { cacheBreaker: `[CACHE_BREAKER: ${injection}]` } : {}),
18    }
19  },
20 )

```

缓存破坏器 (Cache Breaker): 这是一个内部调试机制。由于 Anthropic API 支持 Prompt Caching(提示缓存),修改系统提示会使缓存失效。通过注入一个 `[CACHE_BREAKER: ...]` 标记,可以在不修改实际提示内容的情况下强制刷新缓存。

5.2.3 用户上下文

```

1 export const getUserContext = memoize(
2   async (): Promise<{ [k: string]: string }> => {
3     // 多种条件决定是否加载 CLAUDE.md
4     const shouldDisableClaudeMd =
5       isEnvTruthy(process.env.CLAUDE_CODE_DISABLE_CLAUDE_MDS) ||
6       (isBareMode() && getAdditionalDirectoriesForClaudeMd().
7         length === 0)
8
9     const claudeMd = shouldDisableClaudeMd

```

```
9      ? null
10      : getClaudeMds(filterInjectedMemoryFiles(await
      getMemoryFiles()))
11
12      // 缓存供 yoloClassifier 读取 (避免循环依赖)
13      setCachedClaudeMdContent(claudeMd || null)
14
15      return {
16        ...(claudeMd && { claudeMd }),
17        currentDate: `Today's date is ${getLocalISODate()}.`,
18      }
19    },
20  )
```

CLAUDE.md 机制:

CLAUDE.md 是 Claude Code 的项目配置文件, 类似于 `.editorconfig` 或 `.eslintrc`。开发者可以在项目中放置 CLAUDE.md 文件来定制 Claude 的行为。

文件发现遵循层级:

1. 项目根目录的 CLAUDE.md
2. 当前工作目录的 CLAUDE.md
3. 用户全局配置中的 CLAUDE.md
4. 通过 `--add-dir` 添加的额外目录

5.2.4 系统提示注入控制

```
1 let systemPromptInjection: string | null = null
2
3 export function setSystemPromptInjection(value: string | null):
4   void {
5     systemPromptInjection = value
6     // 注入变更时立即清除上下文缓存
7     getUserContext.cache.clear?().()
8     getSystemContext.cache.clear?().()
9   }
```

当注入内容发生变化时, 所有上下文缓存都会被清除。这确保了下一次查询使用最新的注入内容。

5.3 System Prompt 的构建

5.3.1 构建流程

在 `QueryEngine.ts` 中，System Prompt 通过 `fetchSystemPromptParts` 构建：

```
1  const {
2    defaultSystemPrompt,
3    userContext: baseUserContext,
4    systemContext,
5  } = await fetchSystemPromptParts({
6    tools,
7    mainLoopModel: initialMainLoopModel,
8    additionalWorkingDirectories: Array.from(
9      initialState.toolPermissionContext.
10         additionalWorkingDirectories.keys()
11    ),
12    mcpClients,
13    customSystemPrompt: customPrompt,
14  })
```

5.3.2 Prompt 的分层结构

最终发送给 API 的系统提示由多个部分组成：

```
1
2  System Prompt （完整）
3
4
5    默认系统提示（defaultSystemPrompt）
6    - 角色定义
7    - 基本行为准则
8    - 安全限制
9
10
11
12    工具描述（来自 tools 数组）
13    - 每个工具的名称、输入 schema
14    - 工具使用指南
15
```



```
16
17
18     用户上下文 (userContext)
19     - CLAUDE.md 内容
20     - 当前日期
21     - 记忆文件内容
22
23
24
25     系统上下文 (systemContext)
26     - Git 状态
27     - 缓存破坏器 (如果有)
28
29
30
31     自定义/追加提示
32     - customSystemPrompt (替换)
33     - appendSystemPrompt (追加)
34
35
36
37     协调器上下文 (如果启用)
38     - Worker 可用工具
39     - MCP 服务器列表
40     - Scratchpad 目录
41
42
```

5.3.3 Prompt 缓存优化

Claude Code 非常注重 Prompt Cache 的命中率:

```
1 // 工具池排序以保证缓存稳定性
2 // 服务器的 cache_policy 在最后一个匹配的内置工具后放置缓存断点
3 // 扁平排序会让 MCP 工具与内置工具交错, 使所有下游缓存键失效
4 const byName = (a: Tool, b: Tool) => a.name.localeCompare(b.name)
5 return uniqBy(
6     [...builtInTools].sort(byName).concat(allowedMcpTools.sort(
7         byName)),
```

```
7     'name',  
8 )
```

关键策略：

- 内置工具排在前面，作为连续前缀
- 两组分别按名称排序，保证顺序稳定
- 避免交错：MCP 工具不会插入到内置工具之间

5.4 记忆系统 (Memory)

5.4.1 CLAUDE.md 层级

Claude Code 支持多层级的 CLAUDE.md 文件：

1	~/ .claude/CLAUDE.md	← 全局（用户级别）
2	\$PROJECT_ROOT/CLAUDE.md	← 项目根目录
3	\$PROJECT_ROOT/.claude/CLAUDE.md	← 项目 .claude 目录
4	\$CWD/CLAUDE.md	← 当前工作目录

5.4.2 持久记忆目录 (memdir)

```
1 import { loadMemoryPrompt } from './memdir/memdir.js'
```

除了 CLAUDE.md，Claude Code 还有一个 memdir/ 系统用于存储持久化记忆。这些记忆通过 /memory 命令管理，可以跨会话保留。

5.4.3 自动记忆提取

```
1 import { extractMemories } from './services/extractMemories/'
```

Claude Code 可以自动从对话中提取有价值的记忆信息，并将其保存到记忆目录中。

5.5 附件系统

```
1 import {  
2     createAttachmentMessage,  
3     filterDuplicateMemoryAttachments,
```

```
4   getAttachmentMessages ,  
5   startRelevantMemoryPrefetch ,  
6 } from './utils/attachments.js'
```

在查询循环中，附件（Attachments）作为额外的上下文信息注入：

```
1 // 在循环开始时启动记忆预取  
2 using pendingMemoryPrefetch = startRelevantMemoryPrefetch(  
3   state.messages ,  
4   state.toolUseContext ,  
5 )
```

using 关键字：这里使用了 TypeScript 5.2+ 的 using 声明（Explicit Resource Management）。pendingMemoryPrefetch 实现了 [Symbol.dispose]，在 generator 退出的任何路径上都会自动清理资源并记录遥测。

5.6 嵌套记忆路径去重

```
1 private loadedNestedMemoryPaths = new Set<string>()
```

当子代理（通过 AgentTool 生成）需要加载 CLAUDE.md 时，存在一个微妙的去重问题：

> readFileState 是一个 LRU 缓存，在繁忙的会话中会驱逐条目。因此它的 .has() 检查不足以防止重复注入——同一个 CLAUDE.md 可能被注入数十次。loadedNestedMemoryPaths 提供了一个稳定的去重集合。

5.7 设计启示

5.7.1 分层上下文模型

将上下文分为系统级、项目级、用户级三层，允许在不同粒度上定制行为。这种模式可以推广到任何需要多层配置的 LLM 应用。

5.7.2 缓存友好的设计

Claude Code 在多个层面优化 Prompt Cache：

1. 工具排序稳定
2. 上下文部分按缓存断点分段

3. 变化频率低的内容排在前面（系统提示 > 工具 > 用户上下文 > Git 状态）

5.7.3 快照 vs 实时

Git 状态采用快照语义（对话开始时获取一次），而非实时更新。这是一个深思熟虑的设计：

- 实时更新会导致上下文频繁变化，破坏缓存
- 模型需要通过 BashTool 主动获取最新状态
- 减少了不必要的 Token 消耗

下一章：[工具系统 \(Tool System\)](#)

Chapter 6

工具系统 (Tool System)

6.1 架构概述

工具系统是 Claude Code 的核心特性——它赋予了 LLM ”手脚”，使其能够真正执行软件工程任务。所有工具的注册和管理通过 `src/tools.ts` 完成，而工具的类型定义在 `src/Tool.ts` (约 29,000 行) 中。

6.2 工具类型定义

6.2.1 核心接口

每个工具必须实现一组标准接口：

```
1 export type ToolInputJSONSchema = {
2   type: 'object'
3   properties?: { [x: string]: unknown }
4   [x: string]: unknown
5 }
6
7 // Tool 的核心字段 (从源码推断)
8 interface Tool {
9   name: string // 工具名称 (唯一标识)
10  description: string // 工具描述 (给 LLM 看的)
11  inputSchema: ToolInputJSONSchema // 输入参数 JSON Schema
12  isEnabled(): boolean // 是否启用
13
14  // 权限相关
15  mcpInfo?: { // MCP 工具信息
16    serverName: string
17    toolName: string
18  }
19 }
```

```

20 // 执行相关
21 maxResultSizeChars?: number // 最大结果大小
22 }

```

6.2.2 权限上下文

```

1 export type ToolPermissionContext = DeepImmutable<{
2   mode: PermissionMode
3   additionalWorkingDirectories: Map<string,
4     AdditionalWorkingDirectory>
5   alwaysAllowRules: ToolPermissionRulesBySource
6   alwaysDenyRules: ToolPermissionRulesBySource
7   alwaysAskRules: ToolPermissionRulesBySource
8   isBypassPermissionsModeAvailable: boolean
9   isAutoModeAvailable?: boolean
10  strippedDangerousRules?: ToolPermissionRulesBySource
11  shouldAvoidPermissionPrompts?: boolean
12  awaitAutomatedChecksBeforeDialog?: boolean
13  prePlanMode?: PermissionMode
14 }>

```

关键字段说明:

字段	说明
mode	权限模式: default / plan / bypassPermissions / auto
alwaysAllowRules	始终允许的规则 (按来源分组)
alwaysDenyRules	始终拒绝的规则 (按来源分组)
shouldAvoidPermissionPrompts	后台代理使用: 无法显示 UI 时自动拒绝
prePlanMode	进入计划模式前的权限模式, 退出时恢复

6.3 工具注册表 (tools.ts)

6.3.1 静态工具注册

```

1 import { AgentTool } from './tools/AgentTool/AgentTool.js'
2 import { BashTool } from './tools/BashTool/BashTool.js'
3 import { FileEditTool } from './tools/FileEditTool/FileEditTool.js'

```

```
1  
4 import { FileReadTool } from './tools/FileReadTool/FileReadTool.js'  
5  
5 import { FileWriteTool } from './tools/FileWriteTool/FileWriteTool  
  .js'  
6 import { GlobTool } from './tools/GlobTool/GlobTool.js'  
7 import { GrepTool } from './tools/GrepTool/GrepTool.js'  
8 import { WebFetchTool } from './tools/WebFetchTool/WebFetchTool.js'  
9  
9 import { WebSearchTool } from './tools/WebSearchTool/WebSearchTool  
  .js'  
10 import { SkillTool } from './tools/SkillTool/SkillTool.js'  
11 // ... 更多工具
```

6.3.2 条件工具注册

许多工具通过 Feature Flag 或环境变量条件加载：

```
1 // 仅内部用户可用  
2 const REPLTool = process.env.USER_TYPE === 'ant'  
3   ? require('./tools/REPLTool/REPLTool.js').REPLTool  
4   : null  
5  
6 // 需要 Feature Flag  
7 const SleepTool = feature('PROACTIVE') || feature('KAIRO')  
8   ? require('./tools/SleepTool/SleepTool.js').SleepTool  
9   : null  
10  
11 // 触发器相关工具  
12 const cronTools = feature('AGENT_TRIGGERS')  
13   ? [  
14     require('./tools/ScheduleCronTool/CronCreateTool.js').  
      CronCreateTool,  
15     require('./tools/ScheduleCronTool/CronDeleteTool.js').  
      CronDeleteTool,  
16     require('./tools/ScheduleCronTool/CronListTool.js').  
      CronListTool,  
17   ]  
18   : []
```

```

19
20 // 协调器模式
21 const coordinatorModeModule = feature('COORDINATOR_MODE')
22   ? require('./coordinator/coordinatorMode.js')
23   : null
24
25 // Web 浏览器工具
26 const WebBrowserTool = feature('WEB_BROWSER_TOOL')
27   ? require('./tools/WebBrowserTool/WebBrowserTool.js').
      WebBrowserTool
28   : null

```

6.3.3 懒加载（打破循环依赖）

```

1 // 懒加载以打破循环依赖: tools.ts -> TeamCreateTool -> ... ->
  tools.ts
2 const getTeamCreateTool = () =>
3   require('./tools/TeamCreateTool/TeamCreateTool.js').
      TeamCreateTool
4 const getTeamDeleteTool = () =>
5   require('./tools/TeamDeleteTool/TeamDeleteTool.js').
      TeamDeleteTool
6 const getSendMessageTool = () =>
7   require('./tools/SendMessageTool/SendMessageTool.js').
      SendMessageTool

```

6.3.4 完整工具列表

```

1 export function getAllBaseTools(): Tools {
2   return [
3     AgentTool,           // 子代理生成
4     TaskOutputTool,      // 任务输出
5     BashTool,            // Shell 命令
6     // 嵌入式搜索工具检测
7     ...(hasEmbeddedSearchTools() ? [] : [GlobTool, GrepTool]),
8     ExitPlanModeV2Tool,  // 退出计划模式
9     FileReadTool,        // 文件读取
10    FileEditTool,         // 文件编辑

```



```
11     FileWriteTool,           // 文件写入
12     NotebookEditTool,       // Notebook 编辑
13     WebFetchTool,           // URL 获取
14     TodoWriteTool,          // Todo 写入
15     WebSearchTool,          // 网页搜索
16     TaskStopTool,           // 任务停止
17     AskUserQuestionTool,     // 向用户提问
18     SkillTool,              // 技能执行
19     EnterPlanModeTool,       // 进入计划模式
20     // 条件工具...
21     ...(isTodoV2Enabled()
22         ? [TaskCreateTool, TaskGetTool, TaskUpdateTool, TaskListTool
23           ]
24         : []),
25     ...(isWorktreeModeEnabled()
26         ? [EnterWorktreeTool, ExitWorktreeTool]
27         : []),
28     getSendMessageTool(),    // 代理间消息
29     ...(isAgentSwarmsEnabled()
30         ? [getTeamCreateTool(), getTeamDeleteTool()]
31         : []),
32     BriefTool,               // 简要输出
33     ListMcpResourcesTool,    // 列出 MCP 资源
34     ReadMcpResourceTool,     // 读取 MCP 资源
35     ...(isToolSearchEnabledOptimistic() ? [ToolSearchTool] : []),
36     // ... 更多条件工具
37 }
```

嵌入式搜索工具：在 Anthropic 内部构建中，`bfs` (fast find) 和 `ugrep` (fast grep) 被嵌入到 Bun 二进制文件中。当这些工具可用时，`GlobTool` 和 `GrepTool` 就不需要了——Shell 中的 `find/grep` 已被别名为这些快速工具。

6.4 工具分类详解

6.4.1 文件操作工具

工具	功能	关键特性
----	----	------

FileReadTool	读取文件	支持图片、PDF、Notebook; LRU 缓存
FileWriteTool	创建/覆盖文件	完整文件写入
FileEditTool	部分编辑	字符串替换, 精确匹配
NotebookEditTool	Jupyter Notebook	单元格级别操作
GlobTool	文件搜索	picomatch 模式匹配
GrepTool	内容搜索	ripgrep 后端

6.4.2 执行工具

工具	功能	关键特性
BashTool	Shell 命令	支持 Bash 和 PowerShell
PowerShellTool	PowerShell 命令	Windows 专用
LSPTool	语言服务器	代码智能: 跳转、补全、诊断

6.4.3 网络工具

工具	功能
WebFetchTool	获取 URL 内容
WebSearchTool	执行网页搜索
WebBrowserTool	浏览器自动化 (Feature Flag 控制)

6.4.4 代理与协调工具

工具	功能
AgentTool	生成子代理执行复杂任务
TeamCreateTool	创建团队级并行代理
TeamDeleteTool	删除团队代理
SendMessageTool	代理间消息传递
TaskCreateTool	创建任务
TaskUpdateTool	更新任务状态
TaskStopTool	停止运行中的任务

6.4.5 模式控制工具

工具	功能
EnterPlanModeTool	进入计划模式（只读，不执行）
ExitPlanModeV2Tool	退出计划模式
EnterWorktreeTool	进入 Git Worktree 隔离环境
ExitWorktreeTool	退出 Worktree

6.4.6 特殊工具

工具	功能
SkillTool	执行预定义或自定义技能
ToolSearchTool	延迟工具发现（当工具数量多时）
SleepTool	主动模式下的等待
CronCreateTool	创建定时触发器
SyntheticOutputTool	结构化输出生成
BriefTool	简要输出模式
AskUserQuestionTool	向用户提出交互式问题

6.5 工具过滤机制

6.5.1 拒绝规则过滤

```

1 export function filterToolsByDenyRules<T extends { name: string;
    mcpInfo?: ... }>(
2   tools: readonly T[],
3   permissionContext: ToolPermissionContext,
4 ): T[] {
5   return tools.filter(tool => !getDenyRuleForTool(
        permissionContext, tool))
6 }

```

工具在到达 LLM 之前就被过滤。一个被全面拒绝（blanket deny）的工具根本不会出现在模型的工具列表中。

6.5.2 REPL 模式工具隐藏

```

1 if (isReplModeEnabled()) {
2   const replEnabled = allowedTools.some(tool =>

```

```

3     toolMatchesName(tool, REPL_TOOL_NAME)
4   )
5   if (replEnabled) {
6     // 隐藏原始工具——它们通过 REPL 的 VM 上下文间接可用
7     allowedTools = allowedTools.filter(
8       tool => !REPL_ONLY_TOOLS.has(tool.name)
9     )
10  }
11 }

```

在 REPL 模式下，底层工具（Bash、FileRead 等）被隐藏，因为它们通过 REPL 的 VM 上下文间接可用。

6.5.3 简单模式

```

1  if (isEnvTruthy(process.env.CLAUDE_CODE_SIMPLE)) {
2    const simpleTools: Tool[] = [BashTool, FileReadTool,
      FileEditTool]
3    // 协调器模式下额外添加 AgentTool 和 TaskStopTool
4    if (feature('COORDINATOR_MODE') && coordinatorModeModule?.
      isCoordinatorMode()) {
5      simpleTools.push(AgentTool, TaskStopTool, getSendMessageTool()
6        )
7    }
8    return filterToolsByDenyRules(simpleTools, permissionContext)
9  }

```

简单模式只提供三个基本工具：Bash、读取、编辑。

6.6 工具池组装

```

1  export function assembleToolPool(
2    permissionContext: ToolPermissionContext,
3    mcpTools: Tools,
4  ): Tools {
5    const builtInTools = getTools(permissionContext)
6    const allowedMcpTools = filterToolsByDenyRules(mcpTools,
7      permissionContext)

```

```

8  // 排序以保证 prompt cache 稳定性
9  const byName = (a: Tool, b: Tool) => a.name.localeCompare(b.name
    )
10  return uniqBy(
11    [...builtInTools].sort(byName).concat(allowedMcpTools.sort(
        byName)),
12    'name', // 内置工具在名称冲突时优先
13  )
14 }

```

组装策略：

1. 获取内置工具（经过模式过滤和启用检查）
2. 获取 MCP 工具（经过拒绝规则过滤）
3. 内置工具作为连续前缀排序
4. MCP 工具在后面排序
5. 通过 `uniqBy` 去重（内置工具优先级更高）

6.7 ToolUseContext 深入

每次工具执行时传入的上下文对象包含丰富的运行时信息：

```

1  export type ToolUseContext = {
2    // 配置
3    options: {
4      commands: Command[]
5      tools: Tools
6      mcpClients: MCPServerConnection[]
7      mcpResources: Record<string, ServerResource[]>
8      mainLoopModel: string
9      thinkingConfig: ThinkingConfig
10     agentDefinitions: AgentDefinitionsResult
11     refreshTools?: () => Tools // MCP 服务器中途连接时刷新
12   }
13
14   // 控制
15   abortController: AbortController // 中止信号
16   readFileState: FileStateCache // 文件读取缓存
17
18   // 状态

```

```

19  getAppState(): AppState
20  setAppState(f: (prev: AppState) => AppState): void
21  messages: Message[] // 当前对话消息
22
23  // 限制
24  fileReadingLimits?: { maxTokens?:; maxSizeBytes? }
25  globLimits?: { maxResults? }
26
27  // 权限追踪
28  toolDecisions?: Map<string, { source; decision; timestamp }>
29  localDenialTracking?: DenialTrackingState
30
31  // UI 交互
32  setToolJSX?: SetToolJSXFn // 设置工具 UI
33  addNotification?: (notif) => void // 添加通知
34  sendOSNotification?: (opts) => void // OS 级别通知
35
36  // 子代理
37  agentId?: AgentId // 子代理 ID
38  agentType?: string // 代理类型名称
39 }

```

6.8 设计启示

6.8.1 声明式工具定义

每个工具是自包含的模块——定义自己的输入 Schema、权限模型和执行逻辑。这种模式使得添加新工具极其简单，不需要修改任何中心化的路由逻辑。

6.8.2 条件加载与死代码消除

通过 `feature()` 和 `process.env` 的条件加载，确保未使用的工具代码在构建时被完全移除。这对控制 bundle 大小至关重要。

6.8.3 工具池的缓存感知排序

将内置工具排在前面并保持稳定排序，使得 Prompt Cache 可以跨请求复用。这种“缓存感知”的设计在高频调用场景下可以显著降低成本。

下一章: 命令系统 (Command System)

Chapter 7

命令系统 (Command System)

7.1 概述

命令系统管理用户通过 / 前缀调用的斜杠命令。这些命令提供了直接的用户交互功能，不通过 LLM 处理。命令注册表在 `src/commands.ts` (约 25,000 行) 中管理。

7.2 命令注册

```
1  // commands.ts — 静态导入
2  import commit from './commands/commit.js'
3  import compact from './commands/compact/index.js'
4  import config from './commands/config/index.js'
5  import doctor from './commands/doctor/index.js'
6  import memory from './commands/memory/index.js'
7  import review, { ultrareview } from './commands/review.js'
8  import vim from './commands/vim/index.js'
9  // ... 50+ 命令导入
10
11 // 条件导入 (Feature Flag 控制)
12 const voiceCommand = feature('VOICE_MODE')
13   ? require('./commands/voice/index.js').default
14   : null
15
16 const proactive = feature('PROACTIVE') || feature('KAIROS')
17   ? require('./commands/proactive.js').default
18   : null
19
20 const bridge = feature('BRIDGE_MODE')
21   ? require('./commands/bridge/index.js').default
22   : null
23
```



```
24 // 仅内部用户
25 const agentsPlatform = process.env.USER_TYPE === 'ant'
26   ? require('./commands/agents-platform/index.js').default
27   : null
```

7.3 命令分类

7.3.1 Git 操作

命令	功能
/commit	创建 Git 提交（自动生成提交信息）
/commit-push-pr	提交、推送并创建 PR
/diff	查看当前变更
/pr\{\}_comments	查看 PR 评论
/review	代码审查
/security-review	安全审查

7.3.2 会话管理

命令	功能
/compact	压缩对话上下文
/resume	恢复之前的会话
/share	分享会话
/session	会话管理
/rename	重命名会话
/clear	清除对话历史

7.3.3 配置与状态

命令	功能
/config	设置管理
/memory	持久记忆管理
/doctor	环境诊断
/cost	查看使用费用

<code>/status</code>	查看状态
<code>/usage</code>	查看使用量
<code>/theme</code>	切换主题

7.3.4 工具与扩展

命令	功能
<code>/mcp</code>	MCP 服务器管理
<code>/skills</code>	技能管理
<code>/tasks</code>	任务管理
<code>/init</code>	初始化项目配置
<code>/init-verifiers</code>	初始化验证器

7.3.5 交互模式

命令	功能
<code>/vim</code>	切换 Vim 模式
<code>/keybindings</code>	键绑定配置
<code>/color</code>	颜色设置

7.3.6 认证

命令	功能
<code>/login</code>	登录
<code>/logout</code>	登出
<code>/install-github-app</code>	安装 GitHub App
<code>/install-slack-app</code>	安装 Slack App

7.3.7 跨平台

命令	功能
<code>/desktop</code>	传递到桌面应用
<code>/mobile</code>	传递到移动应用
<code>/ide</code>	IDE 集成

`/teleport` 远程传送会话

7.3.8 Feature Flag 控制的命令

命令	Feature Flag	功能
<code>/voice</code>	<code>VOICE\{}_MODE</code>	语音输入
<code>/proactive</code>	<code>PROACTIVE / KAIROS</code>	主动模式
<code>/bridge</code>	<code>BRIDGE\{}_MODE</code>	IDE 桥接
<code>/assistant</code>	<code>KAIROS</code>	助手模式
<code>/workflows</code>	<code>WORKFLOW\{}_SCRIPTS</code>	工作流管理
<code>/force-snip</code>	<code>HISTORY\{}_SNIP</code>	强制历史片段裁剪

7.4 命令与工具的交叉

命令系统与工具系统有一个有趣的交叉点——`getSlashCommandToolSkills()`：

```
1 import { getSlashCommandToolSkills } from './commands.js'
```

某些斜杠命令可以被注册为“工具技能”，使得 LLM 不仅可以在用户主动调用时使用它们，还可以在认为合适时自主调用。

7.5 远程模式命令过滤

```
1 import { filterCommandsForRemoteMode, getCommands } from './  
  commands.js'
```

在远程模式下，某些不适用的命令（如 `/desktop`、`/vim`）会被自动过滤。

7.6 命令生命周期

```
1 import { notifyCommandLifecycle } from './utils/commandLifecycle.  
  js'  
2  
3 // 在查询循环完成后通知命令完成  
4 for (const uuid of consumedCommandUuids) {  
5   notifyCommandLifecycle(uuid, 'completed')  
6 }
```

每个命令有明确的生命周期追踪：启动 → 执行 → 完成/失败。这支持异步命令的状态追踪和错误报告。

7.7 设计启示

7.7.1 命令 vs 工具的边界

Claude Code 将用户发起的操作（命令）与 LLM 发起的操作（工具）分离，但允许交叉。这种设计提供了清晰的心智模型：

- 命令：用户主动控制，即时反馈
- 工具：LLM 自主调用，异步执行

7.7.2 按需加载

只在需要时加载命令实现（通过 Feature Flag 控制），保持 CLI 的启动速度。

下一章：[权限与安全模型](#)

Part III

安全与资源管理

Chapter 8

权限与安全模型

8.1 概述

Claude Code 的权限系统是其最关键的安全机制。每次 LLM 尝试调用工具时，都必须通过权限检查。这个系统确保 AI 不会在未经用户授权的情况下执行危险操作。

8.2 权限模式

Claude Code 定义了多种权限模式：

模式	行为
default	标准模式——危险操作需要用户确认
plan	计划模式——只允许只读操作，不执行修改
bypassPermissions	绕过权限——所有操作自动批准（危险）
auto	自动模式——使用分类器自动判断是否安全

8.2.1 计划模式

```
1 // EnterPlanModeTool — LLM 可以主动进入计划模式
2 // 进入时保存当前权限模式
3 prePlanMode?: PermissionMode
4
5 // ExitPlanModeV2Tool — 退出时恢复之前的模式
```

计划模式是 Claude Code 的一个独特功能：LLM 可以主动切换到只读模式进行分析和规划，然后退出到执行模式实施变更。

8.2.2 自动模式与分类器

```
1 const autoModeStateModule = feature('TRANSCRIPT_CLASSIFIER')
2   ? require('./utils/permissions/autoModeState.js')
```

```
3 : null
```

自动模式使用**对话分类器** (Transcript Classifier) 分析工具调用的安全性。分类器在权限对话框显示之前运行，可以自动批准被认为安全的操作。

8.3 权限规则

权限规则按来源分组 (ToolPermissionRulesBySource)，支持三种决策：

```
1 type ToolPermissionContext = {
2   alwaysAllowRules: ToolPermissionRulesBySource // 始终允许
3   alwaysDenyRules: ToolPermissionRulesBySource // 始终拒绝
4   alwaysAskRules: ToolPermissionRulesBySource // 始终询问
5 }
```

8.3.1 规则匹配

拒绝规则的匹配支持前缀模式，特别用于 MCP 工具：

```
1 // 可以拒绝整个 MCP 服务器的所有工具
2 // 规则 "mcp__server" 会匹配 "mcp__server__tool1", "
   mcp__server__tool2" 等
```

8.3.2 工具级别过滤

被全面拒绝的工具在模型看到工具列表之前就被移除：

```
1 export function filterToolsByDenyRules(tools, permissionContext) {
2   return tools.filter(tool => !getDenyRuleForTool(
     permissionContext, tool))
3 }
```

8.4 权限检查流程

```
1 LLM 请求调用工具
2
3
4 1. 检查 blanket deny 规则 → 如果匹配，拒绝
```

```
5
6
7 2. 检查 alwaysAllow 规则 → 如果匹配, 允许
8
9
10 3. 检查 alwaysAsk 规则 → 如果匹配, 提示用户
11
12
13 4. 根据 mode 决定:
14     default → 显示权限对话框
15     plan → 拒绝写操作
16     auto → 运行分类器
17         安全 → 允许
18         不确定 → 显示对话框
19     bypassPermissions → 允许
20
21
22 5. 记录决策到 toolDecisions Map
```

8.5 拒绝追踪

```
1 // QueryEngine.ts 中的权限包装
2 const wrappedCanUseTool = async (...) => {
3     const result = await canUseTool(...)
4
5     if (result.behavior !== 'allow') {
6         this.permissionDenials.push({
7             tool_name: sdkCompatToolName(tool.name),
8             tool_use_id: toolUseID,
9             tool_input: input,
10         })
11     }
12
13     return result
14 }
```

所有拒绝都被记录并通过 SDK 事件报告。这支持外部集成（如 VS Code 扩展）显示权限拒绝信息。

8.5.1 本地拒绝追踪

```
1 // 异步子代理的 setAppState 是空操作，  
2 // 没有本地追踪的话拒绝计数永远不会累积  
3 localDenialTracking?: DenialTrackingState
```

当子代理的拒绝次数超过阈值时，会自动回退到向用户提示（prompting）模式。

8.6 后台代理的权限

```
1 shouldAvoidPermissionPrompts?: boolean
```

后台代理（如通过 AgentTool 生成的子代理）无法显示 UI，因此权限提示被自动拒绝。这要求后台代理只执行已预先授权的操作。

8.7 危险权限剥离

```
1 import {  
2   removeDangerousPermissions,  
3   stripDangerousPermissionsForAutoMode,  
4 } from './utils/permissions/permissionSetup.js'
```

某些权限在自动模式下被主动剥离——即使用户配置了 `alwaysAllow`，某些高风险操作在自动模式下仍需要确认。

8.8 沙箱系统

```
1 import { SandboxManager } from './utils/sandbox/sandbox-adapter.js'  
2  
3 // 启动遥测中记录沙箱状态  
4 sandbox_enabled: SandboxManager.isSandboxingEnabled(),  
5 are_unsandboxed_commands_allowed: SandboxManager.  
   areUnsandboxedCommandsAllowed(),  
6 is_auto_bash_allowed_if_sandbox_enabled: SandboxManager.  
   isAutoAllowBashIfSandboxedEnabled(),
```

沙箱系统为 Bash 命令提供额外的隔离层：

- 启用沙箱时，命令在受限环境中执行
- 可以配置是否允许非沙箱命令
- 沙箱内的 Bash 命令可以自动批准

8.9 Scratchpad 目录

```
1 // coordinatorMode.ts
2 if (scratchpadDir && isScratchpadGateEnabled()) {
3     content += `\nScratchpad directory: ${scratchpadDir}\n` +
4         `Workers can read and write here without permission prompts.`
5 }
```

Scratchpad 是一个特殊目录，Worker 代理可以在其中自由读写而无需权限提示。这为多代理协作提供了安全的共享空间。

8.10 设计启示

8.10.1 纵深防御

权限系统体现了纵深防御原则：

1. **工具列表过滤**：拒绝的工具不暴露给模型
2. **调用时检查**：每次调用都经过权限验证
3. **沙箱执行**：即使允许，也在受限环境中执行
4. **追踪与审计**：所有决策都被记录

8.10.2 渐进式信任

从 plan（只读）到 default（需确认）到 auto（自动判断）到 bypassPermissions（全部允许），提供了渐进式的信任层级。

下一章：上下文压缩与 Token 管理

Chapter 9

上下文压缩与 Token 管理

9.1 问题背景

LLM 的上下文窗口是有限的。Claude 的上下文窗口约为 200K tokens，但在长时间的编程会话中，消息历史可以轻松超过这个限制。Claude Code 设计了一套多层压缩系统来解决这个问题。

9.2 五层压缩管线

```
1 原始消息历史
2
3
4 工具结果预算 (Tool Result Budget)
5     控制单个工具输出的大小
6
7 Snip 压缩 (History Snip)
8     移除旧的对话片段
9
10 微压缩 (Microcompact)
11     压缩单个工具输出的冗余
12
13 上下文折叠 (Context Collapse)
14     多轮交互折叠为摘要
15
16 自动压缩 (Auto Compact)
17     接近窗口限制时的全局压缩
18
19 发送到 API
```

9.3 第一层：工具结果预算

```

1 messagesForQuery = await applyToolResultBudget(
2   messagesForQuery,
3   toolUseContext.contentReplacementState,
4   persistReplacements ? records =>
5     void recordContentReplacement(records, toolUseContext.agentId)
6     .catch(logError)
7   : undefined,
8   new Set(
9     toolUseContext.options.tools
10      .filter(t => !Number.isFinite(t.maxResultSizeChars))
11      .map(t => t.name),
12   ),
13 )

```

工作原理：

- 每个工具可以定义 `maxResultSizeChars` 限制
- 超出限制的工具结果被替换为摘要
- 替换决策通过 `contentReplacementState` 持久化
- 支持会话恢复时重建替换状态

与微压缩的关系：工具结果预算在微压缩之前运行。由于缓存微压缩完全通过 `tool_use_id` 操作（不检查内容），内容替换对它是透明的，两者可以干净地组合。

9.4 第二层：Snip 压缩

```

1 if (feature('HISTORY_SNIP')) {
2   const snipResult = snipModule!.snipCompactIfNeeded(
3     messagesForQuery)
4   messagesForQuery = snipResult.messages
5   snipTokensFreed = snipResult.tokensFreed
6   if (snipResult.boundaryMessage) {
7     yield snipResult.boundaryMessage
8   }
9 }

```

Snip 压缩是一种基于”片段”的压缩方式：

- 标识旧的对话片段
- 移除不再需要的旧片段
- 通过 `snipReplay` 回调处理 `snip` 边界
- 释放的 `Token` 数量传递给自动压缩阈值计算

9.4.1 Snip 边界处理

```
1 // QueryEngine 配置中的 snipReplay 回调
2 snipReplay?: (
3   yieldedSystemMsg: Message,
4   store: Message[],
5 ) => { messages: Message[]; executed: boolean } | undefined
```

`snipReplay` 被注入到 `QueryEngine` 中，使得 `Feature Flag` 控制的字符串保持在门控模块内，`QueryEngine` 保持可测试。

9.5 第三层：微压缩

```
1 const microcompactResult = await deps.microcompact(
2   messagesForQuery,
3   toolUseContext,
4   querySource,
5 )
6 messagesForQuery = microcompactResult.messages
```

微压缩处理单个工具输出中的冗余信息，例如：

- 重复的文件列表
- 冗长的命令输出
- 重复的错误信息

9.5.1 缓存微压缩

```
1 const pendingCacheEdits = feature('CACHED_MICROCOMPACT')
2   ? microcompactResult.compactionInfo?.pendingCacheEdits
3   : undefined
```

缓存微压缩是一种优化：使用 `Anthropic API` 的缓存编辑功能，延迟边界消息直到 `API` 响应后才发送（使用实际的 `cache_deleted_input_tokens` 而非估计值）。

9.6 第四层：上下文折叠

```
1 if (feature('CONTEXT_COLLAPSE') && contextCollapse) {
2   const collapseResult = await contextCollapse.
      applyCollapsesIfNeeded(
3     messagesForQuery,
4     toolUseContext,
5     querySource,
6   )
7   messagesForQuery = collapseResult.messages
8 }
```

设计特点：

- 上下文折叠在自动压缩之前运行
- 如果折叠使 Token 数降到自动压缩阈值以下，自动压缩就不会触发
- 折叠视图是**读时投影**，不修改 REPL 的完整历史
- 摘要消息存储在折叠存储中，而非 REPL 消息数组
- `projectView()` 在每次查询入口重放提交日志

9.7 第五层：自动压缩

```
1 import {
2   calculateTokenWarningState,
3   isAutoCompactEnabled,
4   type AutoCompactTrackingState,
5 } from './services/compact/autoCompact.js'
6
7 import { buildPostCompactMessages } from './services/compact/
  compact.js'
```

自动压缩是最后的安全网：

- 当 Token 使用接近上下文窗口限制时自动触发
- 使用 LLM 生成整个对话的摘要
- 替换原有消息历史为摘要 + 最近的消息
- 通过 `/compact` 命令也可以手动触发

9.7.1 响应式压缩

```
1 const reactiveCompact = feature('REACTIVE_COMPACT')
2   ? require('./services/compact/reactiveCompact.js')
3   : null
```

响应式压缩是自动压缩的一个变种——当 API 返回”prompt too long” 错误时触发，是对主动压缩的被动补充。

9.8 Token 计数与估算

```
1 import {
2   doesMostRecentAssistantMessageExceed200k,
3   finalContextTokensFromLastResponse,
4   tokenCountWithEstimation,
5 } from './utils/tokens.js'
```

Token 计数使用两种策略：

1. **精确计数**：从 API 响应的 `usage` 字段获取
2. **估算**：在 API 调用之前，使用估算方法预测 Token 数量

9.9 Token 预算系统

```
1 const budgetTracker = feature('TOKEN_BUDGET')
2   ? createBudgetTracker()
3   : null
```

Token 预算系统（TOKEN_BUDGET Feature Flag）追踪每个回合的 Token 消耗：

```
1 import {
2   getCurrentTurnTokenBudget,
3   getTurnOutputTokens,
4   incrementBudgetContinuationCount,
5 } from './bootstrap/state.js'
6
7 import { createBudgetTracker, checkTokenBudget } from './query/
  tokenBudget.js'
```

9.9.1 任务预算

```

1 // API 的 task_budget (output_config.task_budget)
2 // 与 tokenBudget 的 +500k 自动继续功能不同
3 // `total` 是整个代理回合的预算
4 // `remaining` 从累积 API 使用量中计算
5 taskBudget?: { total: number }

```

任务预算通过 API 的 `output_config.task_budget` 参数控制。压缩后需要重新计算剩余预算：

```

1 // 跨压缩边界的 task_budget.remaining 追踪
2 // 未压缩时服务器可以看到完整历史并自行处理倒计时
3 // 压缩后服务器只看到摘要，会低估消耗
4 // remaining 告诉它压缩前的最终窗口
5 let taskBudgetRemaining: number | undefined = undefined

```

9.10 max__output__tokens 恢复

```

1 const MAX_OUTPUT_TOKENS_RECOVERY_LIMIT = 3
2
3 function isWithheldMaxOutputTokens(msg) {
4   return msg?.type === 'assistant' && msg.apiError === '
      max_output_tokens'
5 }

```

当模型输出被 `max_output_tokens` 截断时：

1. 错误消息被”扣留”，不发送给 SDK 调用方
2. 自动向 API 发送继续请求
3. 最多重试 3 次
4. 只有确认恢复失败后才将错误报告给调用方

9.11 成本追踪

```

1 // cost-tracker.ts
2 export function addToTotalSessionCost(
3   cost: number,

```



```

4   usage: Usage,
5   model: string,
6 ): number {
7   const modelUsage = addToTotalModelUsage(cost, usage, model)
8   addToTotalCostState(cost, modelUsage, model)
9
10  // OpenTelemetry 计数器
11  getCostCounter()?.add(cost, attrs)
12  getTokenCounter()?.add(usage.input_tokens, { ...attrs, type: '
      input' })
13  getTokenCounter()?.add(usage.output_tokens, { ...attrs, type: '
      output' })
14  getTokenCounter()?.add(usage.cache_read_input_tokens ?? 0, {
15    ...attrs, type: 'cacheRead'
16  })
17  getTokenCounter()?.add(usage.cache_creation_input_tokens ?? 0, {
18    ...attrs, type: 'cacheCreation'
19  })
20
21  // 处理 advisor 使用量
22  for (const advisorUsage of getAdvisorUsage(usage)) {
23    const advisorCost = calculateUSDCost(advisorUsage.model,
      advisorUsage)
24    totalCost += addToTotalSessionCost(advisorCost, advisorUsage,
      advisorUsage.model)
25  }
26  return totalCost
27 }

```

成本追踪记录：

- 每个模型的输入/输出 Token
- 缓存读取/创建 Token
- 网页搜索请求数
- Advisor（辅助模型）的使用量
- USD 成本计算

9.11.1 会话成本持久化

```

1 export function saveCurrentSessionCosts(fpsMetrics?: FpsMetrics):

```

```
void {  
2   saveCurrentProjectConfig(current => ({  
3       ...current,  
4       lastCost: getTotalCostUSD(),  
5       lastAPIDuration: getTotalAPIDuration(),  
6       lastTotalInputTokens: getTotalInputTokens(),  
7       lastTotalOutputTokens: getTotalOutputTokens(),  
8       lastSessionId: getSessionId(),  
9       // ...  
10  })  
11 }
```

成本在会话切换前保存到项目配置中，支持跨会话的成本恢复。

9.12 设计启示

9.12.1 多层防御

五层压缩不是冗余设计，每层解决不同粒度的问题。这种“洋葱式”架构确保了即使某一层未能充分压缩，后续层仍能处理。

9.12.2 缓存感知

压缩系统与 Prompt Cache 深度集成。例如，缓存微压缩利用缓存编辑 API 避免不必要的缓存失效。

9.12.3 预算管理

Token 预算和成本追踪的细粒度设计，使得企业用户可以精确控制 AI 的资源消耗。

下一章：多智能体协调 (Coordinator Mode)

Part IV

高级功能

Chapter 10

多智能体协调 (Coordinator Mode)

10.1 概述

协调器模式 (Coordinator Mode) 是 Claude Code 最先进的功能之一。它将 Claude 从单一代理转变为一个**代理编排器**，能够生成多个工作代理 (Worker) 并行处理任务。

10.2 架构设计



10.3 协调器模式的启用

```

1 // coordinatorMode.ts
2 export function isCoordinatorMode(): boolean {
3   if (feature('COORDINATOR_MODE')) {
4     return isEnvTruthy(process.env.CLAUDE_CODE_COORDINATOR_MODE)
5   }
6   return false
7 }

```

需要两个条件同时满足：

1. COORDINATOR_MODE Feature Flag 启用
2. CLAUDE_CODE_COORDINATOR_MODE 环境变量为 truthy

10.4 协调器的系统提示

协调器有一个专用的系统提示（约 400 行），定义了其角色和行为规范：

```

1 export function getCoordinatorSystemPrompt(): string {
2   return `You are Claude Code, an AI assistant that orchestrates
3     software engineering tasks across multiple workers.
4
5   ## 1. Your Role
6   You are a coordinator. Your job is to:
7   - Help the user achieve their goal
8   - Direct workers to research, implement and verify code changes
9   - Synthesize results and communicate with the user
10  - Answer questions directly when possible
11
12  ## 2. Your Tools
13  - AgentTool - Spawn a new worker
14  - SendMessageTool - Continue an existing worker
15  - TaskStopTool - Stop a running worker
16
17  ## 4. Task Workflow
18  | Phase          | Who          | Purpose          |
19  |-----|-----|-----|
20  | Research        | Workers      | Investigate codebase |
21  | Synthesis       | You         | Craft implementation specs |

```

```

22 | Implementation | Workers | Make targeted changes |
23 | Verification   | Workers | Test changes work      |
24 ...`
25 }

```

10.4.1 核心原则

1. **永远综合，不要委派理解：** "Never write 'based on your findings'. You never hand off understanding to another worker."
2. **并行是超能力：** "Parallelism is your superpower. Launch independent workers concurrently."
3. **写操作串行：** "Read-only tasks —parallel freely. Write-heavy tasks —one at a time per set of files."
4. **自包含提示：** "Workers can't see your conversation. Every prompt must be self-contained."

10.5 Worker 工具可用性

```

1 export function getCoordinatorUserContext(
2   mcpClients: ReadonlyArray<{ name: string }>,
3   scratchpadDir?: string,
4 ): { [k: string]: string } {
5   if (!isCoordinatorMode()) return {}
6
7   const workerTools = isEnvTruthy(process.env.CLAUDE_CODE_SIMPLE)
8     ? [BASH_TOOL_NAME, FILE_READ_TOOL_NAME, FILE_EDIT_TOOL_NAME]
9     .sort().join(', ')
10    : Array.from(ASYNC_AGENT_ALLOWED_TOOLS)
11      .filter(name => !INTERNAL_WORKER_TOOLS.has(name))
12      .sort().join(', ')
13
14   let content = `Workers have access to these tools: ${workerTools}
15     `
16
17   // MCP 工具
18   if (mcpClients.length > 0) {
19     const serverNames = mcpClients.map(c => c.name).join(', ')
20     content += `

```

```

        serverNames}`
20     }
21
22     // Scratchpad 目录
23     if (scratchpadDir && isScratchpadGateEnabled()) {
24         content += `\nScratchpad directory: ${scratchpadDir}`
25         content += `\nWorkers can read and write here without
                permission prompts.`
26     }
27
28     return { workerToolsContext: content }
29 }

```

10.5.1 内部工具隔离

```

1  const INTERNAL_WORKER_TOOLS = new Set([
2      TEAM_CREATE_TOOL_NAME,      // Worker 不能创建团队
3      TEAM_DELETE_TOOL_NAME,      // Worker 不能删除团队
4      SEND_MESSAGE_TOOL_NAME,     // Worker 不能发消息给其他 Worker
5      SYNTHETIC_OUTPUT_TOOL_NAME, // 内部使用
6  ])

```

Worker 被阻止使用协调相关的工具，避免层级混乱。

10.6 Worker 通知机制

Worker 完成时，结果以 XML 格式作为用户消息传递给协调器：

```

1  <task-notification>
2      <task-id>{agentId}</task-id>
3      <status>completed|failed|killed</status>
4      <summary>{human-readable status summary}</summary>
5      <result>{agent's final text response}</result>
6      <usage>
7          <total_tokens>N</total_tokens>
8          <tool_uses>N</tool_uses>
9          <duration_ms>N</duration_ms>
10     </usage>
11 </task-notification>

```

10.7 会话模式切换

```
1 export function matchSessionMode(  
2   sessionMode: 'coordinator' | 'normal' | undefined,  
3 ): string | undefined {  
4   if (!sessionMode) return undefined  
5  
6   const currentIsCoordinator = isCoordinatorMode()  
7   const sessionIsCoordinator = sessionMode === 'coordinator'  
8  
9   if (currentIsCoordinator === sessionIsCoordinator) return  
10      undefined  
11  
12   // 动态切换环境变量  
13   if (sessionIsCoordinator) {  
14     process.env.CLAUDE_CODE_COORDINATOR_MODE = '1'  
15   } else {  
16     delete process.env.CLAUDE_CODE_COORDINATOR_MODE  
17   }  
18  
19   return sessionIsCoordinator  
20     ? 'Entered coordinator mode to match resumed session.'  
21     : 'Exited coordinator mode to match resumed session.'  
22 }
```

恢复会话时自动匹配保存时的协调器模式。

10.8 团队代理 (Agent Swarms)

```
1 // tools.ts  
2 ...(isAgentSwarmsEnabled()  
3   ? [getTeamCreateTool(), getTeamDeleteTool()]  
4   : []),
```

团队代理是协调器模式的扩展——允许创建持久化的代理团队，成员之间可以通过 `SendMessageTool` 通信。

10.9 设计启示

10.9.1 提示即架构

协调器的行为完全由其系统提示定义。这意味着：

- 修改协调策略不需要改代码
- 可以 A/B 测试不同的协调策略
- LLM 的能力提升直接改善协调质量

10.9.2 关注点分离

协调器只做高层决策（分解、分派、综合），不直接操作文件系统。Worker 只执行具体任务，不做全局决策。

下一章：[MCP 协议与外部集成](#)

Chapter 11

MCP 协议与外部集成

11.1 什么是 MCP

MCP (Model Context Protocol, 模型上下文协议) 是 Anthropic 推出的开放标准, 用于在 LLM 应用和外部工具/数据源之间建立标准化的连接。Claude Code 是 MCP 的深度实践者。

11.2 MCP 在 Claude Code 中的集成

```
1 Claude Code
2
3     内置工具 (BashTool, FileReadTool, ...)
4
5     MCP 工具
6
7         MCP Server A (如 GitHub)
8             mcp__github__create_issue
9             mcp__github__search_code
10            mcp__github__get_file_contents
11
12         MCP Server B (如数据库)
13             mcp__db__query
14             mcp__db__schema
15
16         MCP Server C (自定义)
17             mcp__custom__my_tool
```

11.3 MCP 服务层

```
1 src/services/mcp/
```

```
2    client.ts          # MCP 客户端管理
3    config.ts          # 配置解析
4    types.ts           # 类型定义
5    claudeai.ts        # Claude AI MCP 配置
6    officialRegistry.ts # 官方 MCP 注册表
7    utils.ts           # 工具函数
8    xaaIdpLogin.ts     # XAA IdP 登录
```

11.3.1 MCP 服务器连接

```
1  type MCPServerConnection = {
2    name: string
3    // 连接状态、工具列表、资源等
4  }
```

11.3.2 MCP 配置

```
1  import {
2    areMcpConfigsAllowedWithEnterpriseMcpConfig,
3    dedupClaudeAiMcpServers,
4    doesEnterpriseMcpConfigExist,
5    filterMcpServersByPolicy,
6    getClaudeCodeMcpConfigs,
7    getMcpServerSignature,
8    parseMcpConfig,
9    parseMcpConfigFromFilePath,
10 } from 'src/services/mcp/config.js'
```

MCP 配置支持多种来源：

- 项目级别配置（.claude/mcp.json）
- 用户级别配置
- 企业级别配置
- Claude AI 远程配置

11.3.3 企业 MCP 管控

```
1  doesEnterpriseMcpConfigExist()    // 是否存在企业配置
```

```

2 filterMcpServersByPolicy()           // 按策略过滤服务器
3 areMcpConfigsAllowedWithEnterpriseMcpConfig() // 企业策略允许性检查

```

11.4 MCP 工具与内置工具的融合

```

1 export function assembleToolPool(
2   permissionContext: ToolPermissionContext,
3   mcpTools: Tools,
4 ): Tools {
5   const builtInTools = getTools(permissionContext)
6   const allowedMcpTools = filterToolsByDenyRules(mcpTools,
7     permissionContext)
8
9   // 内置工具优先，按名称去重
10  return uniqBy(
11    [...builtInTools].sort(byName).concat(allowedMcpTools.sort(
12      byName)),
13    'name',
14  )
15 }

```

冲突解决：当 MCP 工具与内置工具同名时，内置工具优先。

11.5 MCP 资源

除了工具，MCP 还支持“资源”（Resources）——代表可读取的数据源。

```

1 import { ListMcpResourcesTool } from './tools/ListMcpResourcesTool'
2 import { ReadMcpResourceTool } from './tools/ReadMcpResourceTool'

```

工具	功能
ListMcpResourcesTool	列出所有可用的 MCP 资源
ReadMcpResourceTool	读取特定 MCP 资源的内容

11.6 MCP 命令

通过 /mcp 斜杠命令管理 MCP 服务器：

```
1 import mcp from './commands/mcp/index.js'
2 import { registerMcpAddCommand } from 'src/commands/mcp/addCommand
  .js'
3 import { registerMcpXaaIdpCommand } from 'src/commands/mcp/
  xaaIdpCommand.js'
```

11.7 Elicitation 处理

MCP 工具可以通过 -32042 错误码触发 URL 引出 (Elicitation)：

```
1 handleElicitation?: (
2   serverName: string,
3   params: ElicitRequestURLParams,
4   signal: AbortSignal,
5 ) => Promise<ElicitResult>
```

在 SDK 模式下，通过 `structuredIO.handleElicitation` 处理。在 REPL 模式下，使用基于队列的 UI 路径。

11.8 服务器排除

```
1 import {
2   excludeCommandsByServer,
3   excludeResourcesByServer,
4 } from 'src/services/mcp/utils.js'
```

可以按服务器排除特定的命令和资源。

11.9 设计启示

11.9.1 开放协议的价值

MCP 将”工具”的概念从硬编码扩展为可插拔。任何实现 MCP 协议的服务器都可以被 Claude Code 使用，无需修改 Claude Code 本身。

11.9.2 权限统一

MCP 工具与内置工具共享同一套权限系统。`filterToolsByDenyRules` 对两者一视同仁，确保安全策略的一致性。

下一章：终端 UI: React + Ink

Chapter 12

终端 UI: React + Ink

12.1 技术选型

Claude Code 使用 **React + Ink** 构建终端 UI——这是一个非常规但极其优雅的选择。[Ink](#) 是一个使用 React 语法构建命令行界面的框架，将 React 的组件模型和声明式编程带到终端。

12.2 为什么用 React 做终端 UI

传统方案	React + Ink
命令式字符串拼接	声明式组件
手动管理状态更新	React 状态管理
难以复用 UI 逻辑	组件化复用
难以测试	可测试的组件
难以实现复杂布局	Flexbox 布局

12.3 React Canary 版本

```
1 {  
2   "react": "^19.3.0-canary-705268dc-20260409",  
3   "react-reconciler": "^0.34.0-canary-705268dc-20260409"  
4 }
```

Claude Code 使用 React 19 的 canary 版本。这是因为 Ink 需要与 `react-reconciler` 紧密配合，而 reconciler 的稳定版本可能不支持最新的 Ink 特性。

12.4 组件库概览

`src/components/` 目录包含约 140 个组件，覆盖了整个 CLI 界面：

12.4.1 核心组件

组件	功能
REPL.tsx	主 REPL 界面——整个应用的顶层组件
MessageSelector.tsx	消息选择器（选择发送哪些消息给 API）
AgentProgressLine.tsx	代理进度显示
Spinner.tsx	加载动画

12.4.2 Ink 渲染器封装

```
1 src/ink/  
2   renderer.ts      # 自定义 Ink 渲染器  
3   termio/  
4     dec.ts        # 终端控制序列
```

```
1 import { SHOW_CURSOR } from './ink/termio/dec.js'
```

Claude Code 对 Ink 渲染器进行了封装，添加了终端控制序列支持（如光标显示/隐藏）。

12.5 JSX 与工具 UI

工具可以通过 `setToolJSX` 设置自定义的 React UI：

```
1 export type SetToolJSXFn = (  
2   args: {  
3     jsx: React.ReactNode | null  
4     shouldHidePromptInput: boolean  
5     shouldContinueAnimation?: true  
6     showSpinner?: boolean  
7     isLocalJSXCommand?: boolean  
8     isImmediate?: boolean  
9     clearLocalJSX?: boolean  
10  } | null,  
11 ) => void
```

这意味着每个工具执行时可以渲染自己的 UI 组件——例如，文件编辑工具可以显示差异视图，搜索工具可以显示实时结果。

12.6 React Hooks 在 CLI 中的应用

src/hooks/ 目录包含大量 React Hooks:

```
1 src/hooks/  
2     toolPermission/      # 工具权限相关 Hooks  
3     fileSuggestions.ts   # 文件建议  
4     useCanUseTool.ts     # 工具使用权限  
5     ...
```

例如, useCanUseTool Hook 在 REPL 组件中使用, 每次工具调用都通过它进行权限检查。

12.7 全屏界面 (Screens)

```
1 src/screens/  
2     Doctor/             # /doctor 诊断界面  
3     Resume/            # /resume 恢复界面  
4     ...
```

某些命令 (如 /doctor) 需要全屏 UI。这些通过独立的 Screen 组件实现, 接管整个终端。

12.8 FPS 追踪

```
1 import type { FpsMetrics } from './utils/fpsTracker.js'  
2  
3 // 启动遥测中记录  
4 lastFpsAverage: fpsMetrics?.averageFps,  
5 lastFpsLow1Pct: fpsMetrics?.low1PctFps,
```

Claude Code 追踪 UI 的渲染帧率, 记录平均 FPS 和 1

12.9 输出样式

```
1 src/outputStyles/      # 输出样式配置
```

支持不同的输出样式——包括主题、颜色方案等，可通过 `/theme` 命令切换。

12.10 Vim 模式

```
1 src/vim/          # Vim 模式实现
```

Claude Code 甚至内置了 Vim 模式，通过 `/vim` 命令切换。这反映了其目标用户群——习惯在终端中工作的开发者。

12.11 Buddy（伴侣精灵）

```
1 src/buddy/        # 伴侣精灵（彩蛋）
```

一个有趣的彩蛋——Claude Code 包含一个“伴侣精灵”组件，为终端体验增添趣味性。

12.12 设计启示

12.12.1 React 的通用性

React 不仅仅是一个“Web 框架”——它的核心是一个**声明式 UI 编程模型**。通过 `react-reconciler`，这个模型可以适配到任何渲染目标：浏览器 DOM、原生移动端、VR、甚至终端。

12.12.2 组件化的终端 UI

140+ 个组件的规模证明了组件化方法在复杂 CLI 应用中的可行性。传统的字符串拼接方式在这种规模下将变得不可维护。

12.12.3 性能追踪

FPS 追踪在 CLI 应用中很少见，但 Claude Code 的实时流式输出场景确实需要关注渲染性能。

下一章：[Feature Flag](#) 与条件编译

Part V

运营与扩展

Chapter 13

Feature Flag 与条件编译

13.1 双层 Feature Flag 系统

Claude Code 使用两层 Feature Flag 系统，分别在编译时和运行时控制功能：

层次	机制	作用时机	管理方式
编译时	bun:bundle	Build/Run	代码中静态定义
运行时	GrowthBook	应用启动后	远程控制面板

13.2 编译时 Feature Flag (bun:bundle)

13.2.1 工作原理

```
1 import { feature } from 'bun:bundle'
2
3 // 编译时：如果 VOICE_MODE 未启用，整个分支被移除
4 const voiceCommand = feature('VOICE_MODE')
5   ? require('./commands/voice/index.js').default
6   : null
```

Bun 的 bundler 在编译时将 feature() 调用替换为 true 或 false 常量，然后通过死代码消除 (Tree Shaking) 移除不可达的代码分支。

13.2.2 开发环境 Polyfill

```
1 // plugins/bunBundleDev.ts
2 plugin({
3   name: 'bun-bundle-dev',
4   setup(build) {
5     build.module('bun:bundle', () => ({
6       exports: {
```

```

7         feature(flag: string): boolean {
8             return enabledFlags.has(flag)
9         },
10    },
11    loader: 'object',
12  )))
13 },
14 })

```

开发时通过环境变量控制:

```
1 FEATURE_FLAGS=KAIROS,VOICE_MODE bun run src/main.tsx
```

13.2.3 已知 Feature Flag 列表

从源码中提取的所有编译时 Feature Flag:

Flag	功能	影响范围
PROACTIVE	主动模式	SleepTool, proactive 命令
KAIROS	助手模式	assistant 模块, SendUserFileTool, PushNotif
KAIROS\{}_BRIEF	简要模式	brief 命令
KAIROS\{}_PUSH\{}_NOTIFICATION	推送通知	PushNotificationTool
KAIROS\{}_GITHUB\{}_WEBHOOKS	GitHub Webhook	SubscribePRTTool
BRIDGE\{}_MODE	IDE 桥接	bridge 命令, remoteControlServer
DAEMON	守护进程	remoteControlServer 命令
VOICE\{}_MODE	语音输入	voice 命令
AGENT\{}_TRIGGERS	代理触发器	CronCreate/Delete/ListTool
AGENT\{}_TRIGGERS\{}_REMOTE	远程触发	RemoteTriggerTool
MONITOR\{}_TOOL	监控工具	MonitorTool
COORDINATOR\{}_MODE	协调器模式	coordinatorMode 模块
HISTORY\{}_SNIP	历史裁剪	SnipTool, snipCompact
CONTEXT\{}_COLLAPSE	上下文折叠	CtxInspectTool, contextCollapse
CACHED\{}_MICROCOMPACT	缓存微压缩	microcompact 缓存编辑
REACTIVE\{}_COMPACT	响应式压缩	reactiveCompact
TOKEN\{}_BUDGET	Token 预算	tokenBudget 追踪
OVERFLOW\{}_TEST\{}_TOOL	溢出测试	OverflowTestTool
TERMINAL\{}_PANEL	终端面板	TerminalCaptureTool
WEB\{}_BROWSER\{}_TOOL	浏览器工具	WebBrowserTool

TRANSCRIPT\{}_CLASSIFIER	对话分类器	autoModeState
BREAK\{}_CACHE\{}_COMMAND	缓存破坏	systemPromptInjection
UDS\{}_INBOX	UDS 收件箱	ListPeersTool
WORKFLOW\{}_SCRIPTS	工作流脚本	WorkflowTool
BG\{}_SESSIONS	后台会话	taskSummary
EXPERIMENTAL\{}_SKILL\{}_SEARCH	技能搜索	skillSearch
TEMPLATES	模板	job classifier
CCR\{}_REMOTE\{}_SETUP	远程设置	remote-setup 命令

13.3 运行时 Feature Flag (GrowthBook)

```

1 import {
2   initializeGrowthBook,
3   refreshGrowthBookAfterAuthChange,
4   getFeatureValue_CACHED_MAY_BE_STALE,
5   checkStatsigFeatureGate_CACHED_MAY_BE_STALE,
6 } from './services/analytics/growthbook.js'

```

13.3.1 缓存策略

函数名 `_CACHED_MAY_BE_STALE` 暗示了一个重要设计：

- GrowthBook 值在启动时获取并缓存
- 缓存值可能过期但仍被使用
- 这避免了每次检查时的网络请求
- 认证变更后会刷新

13.3.2 使用场景

```

1 // Scratchpad 功能门
2 function isScratchpadGateEnabled(): boolean {
3   return checkStatsigFeatureGate_CACHED_MAY_BE_STALE('
4     tengu_scratch')
5 }

```

运行时 Flag 用于：

- A/B 测试新功能

- 灰度发布
- 远程紧急开关
- 用户分群

13.4 环境变量控制

除了两层 Feature Flag，Claude Code 还大量使用环境变量：

```
1 // 用户类型
2 process.env.USER_TYPE === 'ant'           // 内部用户
3
4 // 功能开关
5 process.env.CLAUDE_CODE_COORDINATOR_MODE // 协调器模式
6 process.env.CLAUDE_CODE_SIMPLE           // 简单模式
7 process.env.CLAUDE_CODE_REMOTE           // 远程模式
8 process.env.CLAUDE_CODE_VERIFY_PLAN      // 验证计划
9 process.env.ENABLE_LSP_TOOL               // LSP 工具
10 process.env.CLAUDE_CODE_DISABLE_CLAUDE_MDS // 禁用 CLAUDE.md
11
12 // 测试
13 process.env.NODE_ENV === 'test'           // 测试模式
```

13.5 构建时常量

```
1 # bunfig.toml
2 [define]
3 "MACRO.VERSION" = '"1.0.0-dev"'
4 "MACRO.BUILD_TIME" = ''
5 "MACRO.PACKAGE_URL" = '"@anthropic-ai/claude-code"'
```

MACRO.* 常量在构建时由 Bun 的 `--define` 注入，用于嵌入版本号、构建时间等信息。

13.6 内部 vs 外部构建

```
1 // 代码中有许多这样的条件
2 if ("external" !== 'ant') {
3   // 外部构建的行为
```

```
4 }
```

字符串 "external" 在内部构建中被替换为 "ant"。这通过 Bun 的 define 系统实现：

- **外部构建**： "external" !== 'ant' → true → 执行外部逻辑
- **内部构建**： "ant" !== 'ant' → false → 跳过（执行内部逻辑）

13.7 设计启示

13.7.1 编译时 vs 运行时

两层 Flag 服务于不同目的：

- **编译时**：减小 bundle 大小，消除未使用代码
- **运行时**：支持远程控制和灰度发布

13.7.2 命名规范

_CACHED_MAY_BE_STALE 后缀是优秀的 API 设计——函数名本身就传达了使用注意事项。

下一章：[IDE 桥接与远程会话](#)

Chapter 14

IDE 桥接与远程会话

14.1 桥接系统概述

Claude Code 通过桥接系统（Bridge System）实现与 IDE 的双向通信。支持 VS Code 和 JetBrains 等主流 IDE。



14.2 桥接架构

```
1 src/bridge/
2     bridgeMain.ts           # 桥主循环
3     bridgeMessaging.ts      # 消息协议
4     bridgePermissionCallbacks.ts # 权限回调
5     replBridge.ts           # REPL 会话桥
6     jwtUtils.ts             # JWT 认证
7     sessionRunner.ts        # 会话执行管理
```

14.2.1 认证

桥接使用 JWT（JSON Web Token）进行身份验证，确保只有授权的 IDE 扩展可以连接到 CLI 进程。

14.2.2 消息协议

基于 JSON-RPC 的双向消息协议，支持：

- IDE → CLI：用户输入、命令请求
- CLI → IDE：流式响应、工具调用通知、权限请求

14.2.3 权限回调

```
1 // bridgePermissionCallbacks.ts
2 // IDE 接管权限对话框的显示
3 // CLI 发送权限请求 → IDE 显示对话框 → 用户决策 → 回传给 CLI
```

14.3 远程会话

```
1 src/remote/
2   RemoteSessionManager.ts    # 远程会话管理器
3   ...
```

Claude Code 支持远程会话——在一台机器上运行 CLI，从另一台机器（或移动设备）访问：

```
1 import { createRemoteSessionConfig } from './remote/
   RemoteSessionManager.js'
2 import { getRemoteSessionUrl } from './constants/product.js'
```

14.3.1 Teleport

```
1 import { teleportToRemoteWithErrorHandling } from './utils/
   teleport.js'
```

“Teleport” 功能允许将本地会话传送到远程环境：

- 验证 Git 状态
- 处理分支切换
- 恢复远程会话
- 处理仓库不匹配

14.3.2 跨平台传递

```
1 import desktop from './commands/desktop/index.js' // → 桌面应用
2 import mobile from './commands/mobile/index.js'   // → 移动应用
```

14.4 服务器模式

```
1 src/server/
2   createDirectConnectSession.ts
3   ...
```

```
1 import { createDirectConnectSession, DirectConnectError } from './
  server/createDirectConnectSession.js'
```

Claude Code 可以作为服务器运行，接受来自其他客户端的直接连接。

14.5 LSP 集成

```
1 import { initializeLspServerManager } from './services/lsp/manager
  .js'
```

Claude Code 集成了 LSP (Language Server Protocol)，提供代码智能功能：

- 符号跳转
- 类型信息
- 代码补全建议
- 诊断信息

LSP 通过 LSPTool 暴露给 LLM，使其能够利用语言服务器的能力进行更精确的代码分析。

14.6 设计启示

14.6.1 CLI 优先，IDE 其次

Claude Code 的核心是 CLI 工具，IDE 集成通过桥接层实现。这种架构确保了：

- CLI 可以独立使用

- IDE 扩展是可选的增强
- 不同 IDE 共享相同的后端逻辑

下一章: [成本追踪与遥测](#)

Chapter 15

成本追踪与遥测

15.1 成本追踪系统

Claude Code 实现了精细的成本追踪系统，让用户实时了解 API 使用费用。

15.1.1 核心数据结构

```
1 type ModelUsage = {
2   inputTokens: number
3   outputTokens: number
4   cacheReadInputTokens: number
5   cacheCreationInputTokens: number
6   webSearchRequests: number
7   costUSD: number
8   contextWindow: number
9   maxOutputTokens: number
10 }
```

15.1.2 成本计算

```
1 export function addToTotalSessionCost(
2   cost: number,
3   usage: Usage,
4   model: string,
5 ): number {
6   // 1. 更新模型使用量统计
7   const modelUsage = addToTotalModelUsage(cost, usage, model)
8   addToTotalCostState(cost, modelUsage, model)
9
10  // 2. OpenTelemetry 计数器
11  getCostCounter()?.add(cost, attrs)
```

```

12   getTokenCounter()?.add(usage.input_tokens, { ...attrs, type: '
      input' })
13   getTokenCounter()?.add(usage.output_tokens, { ...attrs, type: '
      output' })
14   getTokenCounter()?.add(usage.cache_read_input_tokens ?? 0, {
15     ...attrs, type: 'cacheRead'
16   })
17
18   // 3. 递归处理 Advisor 使用量
19   for (const advisorUsage of getAdvisorUsage(usage)) {
20     const advisorCost = calculateUSDCost(advisorUsage.model,
      advisorUsage)
21     totalCost += addToTotalSessionCost(advisorCost, advisorUsage,
      advisorUsage.model)
22   }
23
24   return totalCost
25 }

```

15.1.3 Fast Mode 追踪

```

1  const attrs = isFastModeEnabled() && usage.speed === 'fast'
2    ? { model, speed: 'fast' }
3    : { model }

```

Fast Mode（快速模式）下的使用量单独标记。

15.1.4 格式化输出

```

1  export function formatTotalCost(): string {
2    return chalk.dim(
3      `Total cost:           ${costDisplay}\n` +
4      `Total duration (API):  ${formatDuration(getTotalAPIDuration())}
      \n` +
5      `Total duration (wall): ${formatDuration(getTotalDuration())}\n`
      +
6      `Total code changes:   ${getTotalLinesAdded()} lines added, `
      +
7      `${getTotalLinesRemoved()} lines removed\n` +

```

```
8     `${modelUsageDisplay}`  
9   )  
10 }
```

输出示例:

```
1 Total cost:          $0.1234  
2 Total duration (API): 12.3s  
3 Total duration (wall): 45.6s  
4 Total code changes:  42 lines added, 13 lines removed  
5 Usage by model:  
6     claude-sonnet-4-20260514: 15,234 input, 3,456 output,  
    12,000 cache read, 2,000 cache write ($0.0234)
```

15.2 会话成本持久化

```
1 export function saveCurrentSessionCosts(fpsMetrics?: FpsMetrics):  
    void {  
2     saveCurrentProjectConfig(current => ({  
3         ...current,  
4         lastCost: getTotalCostUSD(),  
5         lastAPIDuration: getTotalAPIDuration(),  
6         lastTotalInputTokens: getTotalInputTokens(),  
7         lastTotalOutputTokens: getTotalOutputTokens(),  
8         lastSessionId: getSessionId(),  
9         lastFpsAverage: fpsMetrics?.averageFps,  
10        lastFpsLow1Pct: fpsMetrics?.low1PctFps,  
11        lastModelUsage: Object.fromEntries(  
12            Object.entries(getModelUsage()).map(([model, usage]) => [  
13                model, {  
14                    inputTokens: usage.inputTokens,  
15                    outputTokens: usage.outputTokens,  
16                    cacheReadInputTokens: usage.cacheReadInputTokens,  
17                    cacheCreationInputTokens: usage.cacheCreationInputTokens,  
18                    webSearchRequests: usage.webSearchRequests,  
19                    costUSD: usage.costUSD,  
20                }])  
19        })  
20    ),
```

```
21     })))
22 }
```

成本在会话切换前保存，支持通过 `/resume` 恢复成本数据。

15.3 OpenTelemetry 集成

```
1 {
2   "@opentelemetry/api": "^1.9.1",
3   "@opentelemetry/api-logs": "^0.214.0",
4   "@opentelemetry/core": "^2.6.1",
5   "@opentelemetry/resources": "^2.6.1",
6   "@opentelemetry/sdk-logs": "^0.214.0",
7   "@opentelemetry/sdk-metrics": "^2.6.1",
8   "@opentelemetry/sdk-trace-base": "^2.6.1",
9   "@opentelemetry/semantic-conventions": "^1.40.0"
10 }
```

Claude Code 集成了完整的 OpenTelemetry 全家桶：

- **Traces**：请求链路追踪
- **Metrics**：Token 计数器、成本计数器
- **Logs**：结构化日志

15.3.1 懒加载优化

```
1 // OpenTelemetry (~400KB) 和 gRPC (~700KB) 通过动态 import() 延迟
   加载
2 // 只在实际需要发送遥测时才加载
```

15.4 分析事件

所有遥测事件使用 `tengu_` 前缀（Claude Code 内部代号“天狗”）：

```
1 logEvent('tengu_startup_telemetry', {
2   is_git: isGit,
3   worktree_count: worktreeCount,
4   gh_auth_status: ghAuthStatus,
```



```
5   sandbox_enabled: SandboxManager.isSandboxingEnabled(),
6   // ...
7 })
8
9 logEvent('tengu_managed_settings_loaded', {
10   keyCount: allKeys.length,
11   keys: allKeys.join(','),
12 })
13
14 logEvent('tengu_coordinator_mode_switched', {
15   to: sessionMode,
16 })
17
18 logEvent('tengu_advisor_tool_token_usage', {
19   advisor_model: advisorUsage.model,
20   input_tokens: advisorUsage.input_tokens,
21   cost_usd_micros: Math.round(advisorCost * 1_000_000),
22 })
```

15.4.1 安全类型标记

```
1 type AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS =
   string
```

这个冗长的类型名是一个**代码审查辅助工具**：每次将字符串作为分析元数据传递时，开发者必须显式将其断言为这个类型，确认该值不包含代码或文件路径（隐私保护）。

15.5 诊断日志

```
1 import { logForDiagnosticsNoPII } from './utils/diagLogs.js'
2
3 logForDiagnosticsNoPII('info', 'git_status_started')
4 logForDiagnosticsNoPII('info', 'git_commands_completed', {
5   duration_ms: Date.now() - gitCmdsStart,
6   status_length: status.length,
7 })
8 logForDiagnosticsNoPII('error', 'git_status_failed', {
9   duration_ms: Date.now() - startTime,
```

```
10 })
```

NoPII (No Personally Identifiable Information) 后缀表明这些日志不包含个人可识别信息。

15.6 启动性能分析

```
1 import { profileCheckpoint, profileReport } from './utils/  
    startupProfiler.js'  
2  
3 profileCheckpoint('main_tsx_entry')  
4 // ... 模块加载 ...  
5 profileCheckpoint('main_tsx_imports_loaded')
```

启动分析器在关键点放置检查点，测量每个阶段的耗时。

15.7 设计启示

15.7.1 隐私优先的遥测

`AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS` 和 `logForDiagnosticsNoPII` 展示了如何在类型系统层面强制执行隐私策略。

15.7.2 多维成本追踪

按模型、按 Token 类型（输入/输出/缓存读/缓存写）、按功能（主循环/Advisor）的多维追踪，使得成本分析可以精确到每个维度。

下一章：插件与技能系统

Chapter 16

插件与技能系统

16.1 插件系统

Claude Code 的插件系统允许通过第三方扩展来增强功能。

16.1.1 架构

```
1 src/plugins/
2     bundled/          # 内置插件
3         index.ts      # 内置插件初始化
4     ...
5
6 src/services/plugins/
7     pluginLoader.ts    # 插件加载器
8     pluginCliCommands.ts # 插件 CLI 命令
9
10 src/utils/plugins/
11     installedPluginsManager.ts # 已安装插件管理
12     managedPlugins.ts         # 托管插件
13     pluginDirectories.ts      # 插件目录
14     cacheUtils.ts             # 缓存工具
15     orphanedPluginFilter.ts   # 孤立插件过滤
16     pluginLoader.ts           # 插件加载逻辑
```

16.1.2 插件加载

```
1 import { initBuiltinPlugins } from './plugins/bundled/index.js'
2 import { loadAllPluginsCacheOnly } from './utils/plugins/
   pluginLoader.js'
3
4 // 启动时初始化内置插件
```

```
5 initBuiltinPlugins()  
6  
7 // 缓存优先加载所有插件  
8 const { enabled, errors } = await loadAllPluginsCacheOnly()
```

16.1.3 插件管理命令

```
1 import {  
2   VALID_INSTALLABLE_SCOPES,  
3   VALID_UPDATE_SCOPES,  
4 } from './services/plugins/pluginCliCommands.js'
```

支持安装、更新、删除插件，以及按作用域（全局/项目/用户）管理。

16.1.4 版本管理

```
1 import { initializeVersionedPlugins } from './utils/plugins/  
   installedPluginsManager.js'  
2 import { cleanupOrphanedPluginVersionsInBackground } from './utils  
   /plugins/cacheUtils.js'
```

插件支持版本管理，并在后台清理孤立版本。

16.1.5 托管插件

```
1 import { getManagedPluginNames } from './utils/plugins/  
   managedPlugins.js'
```

”托管插件”是由 Anthropic 或企业管理员配置的插件，用户无法自行修改。

16.2 技能系统

技能（Skills）是可复用的工作流定义，类似于”预设的操作序列”。

16.2.1 架构

```
1 src/skills/  
2   bundled/           # 内置技能
```

```
3         index.ts           # 内置技能初始化
4         ...
```

16.2.2 内置技能初始化

```
1 import { initBundledSkills } from './skills/bundled/index.js'
```

16.2.3 技能工具

```
1 import { SkillTool } from './tools/SkillTool/SkillTool.js'
```

SkillTool 是 LLM 调用技能的入口。通过它，模型可以执行预定义的工作流，如 `/commit`（提交代码）、`/verify`（验证变更）等。

16.2.4 技能发现

```
1 // 技能发现预取——每次迭代运行
2 const pendingSkillPrefetch = skillPrefetch?.
    startSkillDiscoveryPrefetch(
3     null,
4     messages,
5     toolUseContext,
6 )
```

技能发现在查询循环的每次迭代中运行，与模型流式输出和工具执行并行。这替代了之前在 `getAttachmentMessages` 中的阻塞式发现路径。

16.2.5 技能搜索

```
1 const clearSkillIndexCache = feature('EXPERIMENTAL_SKILL_SEARCH')
2   ? require('./services/skillSearch/localSearch.js').
    clearSkillIndexCache
3   : null
```

实验性技能搜索功能支持在大量技能中进行全文搜索。

16.2.6 斜杠命令技能

```
1 import { getSlashCommandToolSkills } from './commands.js'
```

某些斜杠命令可以注册为技能，使得 LLM 可以自主调用它们。

16.3 技能变更检测

```
1 import { skillChangeDetector } from './utils/skills/skillChangeDetector.js'
```

当技能文件发生变更时，系统能检测并重新加载，无需重启。

16.4 遥测

```
1 import { logSkillsLoaded } from './utils/telemetry/skillLoadedEvent.js'
2 import { logPluginsEnabledForSession } from './utils/telemetry/pluginTelemetry.js'
3 import { logPluginLoadErrors } from './utils/telemetry/pluginTelemetry.js'
4
5 // 记录加载的技能
6 void logSkillsLoaded(getCwd(), getContextWindowForModel(model, getSdkBetas()))
7
8 // 记录启用的插件
9 logPluginsEnabledForSession(enabled, managedNames, getPluginSeedDirs())
10
11 // 记录插件加载错误
12 logPluginLoadErrors(errors, managedNames)
```

16.5 设计启示

16.5.1 技能 vs 工具 vs 命令

Claude Code 的三层扩展模型：

层次	目的	调用者	粒度
工具 (Tool)	原子操作	LLM	低（读文件、执行命令）
技能 (Skill)	工作流	LLM / 用户	中（提交、审查）
命令 (Command)	用户操作	用户	高（配置、诊断）

16.5.2 渐进式扩展

- 内置插件/技能提供基础功能
- 用户可以添加自定义技能
- 企业可以通过托管插件统一配置
- 第三方可以发布开源插件

下一章：[设计模式与工程实践总结](#)

Part VI

总结

Chapter 17

设计模式与工程实践总结

17.1 架构级模式

17.1.1 并行预取 (Parallel Prefetch)

Claude Code 最具特色的架构模式是**利用模块加载时间进行并行预取**：

```
1 // 在 import 语句之前启动异步 I/O
2 profileCheckpoint('main_tsx_entry');
3 startMdmRawRead();           // 启动 MDM 子进程
4 startKeychainPrefetch();    // 启动钥匙串读取
5
6 // 后续的 import 语句需要 ~135ms
7 // 此时 MDM 和 Keychain 读取在后台进行
8 import chalk from 'chalk';
9 import React from 'react';
10 // ...
```

原则：识别“免费”的等待时间，将 I/O 操作前移到该窗口。

17.1.2 分层延迟初始化

- 1 阶段 1：模块求值前 → 启动异步 I/O
- 2 阶段 2：模块求值中 → 自动完成（JS 引擎）
- 3 阶段 3：首次渲染前 → 必要的同步初始化
- 4 阶段 4：首次渲染后 → 延迟预取（用户不感知）

17.1.3 AsyncGenerator 驱动的流程式处理

```
1 async function* query(params): AsyncGenerator<StreamEvent |
    Message> {
2   while (true) {
```

```

3     yield { type: 'stream_request_start' }
4     // ... API 调用 & 工具执行 ...
5     yield assistantMessage
6     // ... 判断是否继续 ...
7   }
8 }

```

使用 AsyncGenerator 实现流式数据管道，允许调用方实时消费每个事件。

17.2 代码组织模式

17.2.1 条件导入与死代码消除

```

1 const SleepTool = feature('PROACTIVE')
2   ? require('./tools/SleepTool/SleepTool.js').SleepTool
3   : null

```

规律：所有可选功能都使用三元表达式 + require() 而非顶层 import。这使得 Bun 的 bundler 可以在编译时完全移除未使用的模块。

17.2.2 懒加载打破循环依赖

```

1 // 不使用顶层 import (会创建循环依赖)
2 const getTeamCreateTool = () =>
3   require('./tools/TeamCreateTool/TeamCreateTool.js').
    TeamCreateTool

```

规律：将导入包装在函数中，只在实际调用时执行。搭配 TypeScript 的 typeof import(...) 保留类型安全。

17.2.3 Memoize 模式

```

1 export const getSystemContext = memoize(async () => {
2   // 只执行一次的重量级操作
3   const gitStatus = await getGitStatus()
4   return { gitStatus }
5 })

```

对于整个会话期间不变的数据（Git 状态、CLAUDE.md 内容），使用 `memoize` 缓存结果。

17.2.4 清除缓存的信号

```
1 export function setSystemPromptInjection(value: string | null):  
    void {  
2     systemPromptInjection = value  
3     getUserContext.cache.clear?()    // 清除 memoize 缓存  
4     getSystemContext.cache.clear?()  
5 }
```

当底层数据变化时，手动清除 `memoize` 缓存。

17.3 类型系统模式

17.3.1 DeepImmutable

```
1 export type ToolPermissionContext = DeepImmutable<{  
2     mode: PermissionMode  
3     alwaysAllowRules: ToolPermissionRulesBySource  
4     // ...  
5 }>
```

使用 `DeepImmutable` 确保权限上下文在传递过程中不被意外修改。

17.3.2 自文档化类型名

```
1 type AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS =  
    string
```

类型名本身就是一条检查清单。每次使用时，开发者必须 `pause` 并确认。

17.3.3 品牌类型 (Branded Types)

```
1 import { asSessionId } from './types/ids.js'  
2 import type { AgentId } from './types/ids.js'
```

使用品牌类型防止 ID 类型之间的混用。

17.4 错误处理模式

17.4.1 恢复循环

```
1  const MAX_OUTPUT_TOKENS_RECOVERY_LIMIT = 3
2
3  // 自动恢复，最多重试 3 次
4  // 恢复期间扣留错误消息，避免影响 SDK 调用方
```

17.4.2 优雅降级

```
1  const truncatedStatus = status.length > MAX_STATUS_CHARS
2    ? status.substring(0, MAX_STATUS_CHARS) +
3      '\n... (truncated. Use BashTool for full info)'
4    : status
```

超出限制时截断并提供获取完整信息的替代方案。

17.4.3 分类重试

```
1  import { categorizeRetryableAPIError } from './services/api/errors
   .js'
2  import { FallbackTriggeredError } from './services/api/withRetry.
   js'
```

API 错误被分类为可重试和不可重试，支持自动重试和模型 fallback。

17.5 安全模式

17.5.1 纵深防御

- 1 1. 工具列表过滤 → 不暴露被拒工具
- 2 2. 调用时权限检查 → 每次操作验证
- 3 3. 沙箱执行 → 受限环境
- 4 4. 审计追踪 → 记录所有决策

17.5.2 隐私优先

```
1 logForDiagnosticsNoPII(...)      // 日志不含 PII
2 _I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS // 类型级别隐私检查
```

17.5.3 信任边界

```
1 if (!hasTrust) {
2     // 不执行 Git 命令 (可能通过 hooks 执行任意代码)
3 }
```

17.6 性能模式

17.6.1 并行化

```
1 // 5 个 Git 命令并行
2 const [branch, mainBranch, status, log, userName] = await Promise.
    all([...])
3
4 // 启动时并行预取
5 startMdmRawRead()
6 startKeychainPrefetch()
7 void getSystemContext()
```

17.6.2 缓存感知排序

```
1 // 内置工具排在前面, 保持稳定排序
2 // 使 Prompt Cache 跨请求复用
3 return uniqBy(
4     [...builtInTools].sort(byName).concat(allowedMcpTools.sort(
5         byName)),
6     'name',
7 )
```

17.6.3 延迟加载重量级模块

```
1 OpenTelemetry (~400KB) → 动态 import()
2 gRPC (~700KB) → 动态 import()
```

17.7 项目规模管理

一个 512,000+ 行的 TypeScript 项目面临的挑战和应对：

挑战	Claude Code 的应对
循环依赖	懒加载 <code>require()</code> 函数
启动速度	并行预取 + 延迟初始化
Bundle 大小	Feature Flag + 死代码消除
类型安全	严格模式 + 品牌类型
配置演进	版本化迁移系统
隐私合规	类型级别隐私标记

17.8 总结

Claude Code 的源码揭示了构建工业级 AI 编程助手的复杂度。它不仅仅是一个“调用 API 的 CLI 工具”，而是一个包含以下系统的完整平台：

1. **查询引擎**：多层压缩管线、自动恢复、Token 预算
2. **工具系统**：40+ 工具、权限模型、缓存感知
3. **命令系统**：50+ 命令、生命周期管理
4. **协调器**：多代理编排、并行执行
5. **UI 框架**：React + Ink 的 140+ 组件
6. **安全层**：多模式权限、沙箱、审计
7. **扩展系统**：插件、技能、MCP
8. **遥测**：OpenTelemetry、成本追踪、性能分析

这些系统的设计模式——并行预取、缓存感知、分层压缩、纵深防御——不仅适用于 AI 编程助手，也可以推广到任何需要将 LLM 深度集成到软件工程流程中的应用。

全书完

> 本书基于 `szylover/claude-code` 仓库中的公开源码撰写。> 所有原始源码的知识产权归 Anthropic 所有。